16

High performance computing and parallelism

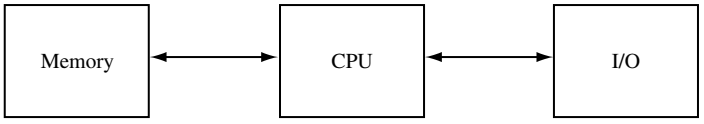
16.1 Introduction

It is not necessary to recall the dramatic increase in computer speed and the drop in cost of hardware over the last two decades. Today, anyone can buy a computer with which all of the programs in this book can be executed within a reasonable time – typically a few seconds to a few hours.

On the other hand, if there is one conclusion to be drawn from the enormous amount of research in computational physics, it should be that for most physical problems, a realistic treatment, one without severe approximations, is still not within reach. Quantum many-particle problems, for example, can only be treated if the correlations are treated in an approximate way (this does not hold for quantum Monte Carlo techniques, but there we suffer from minus-sign problems when treating fermions; see Chapter 12). It is easy to extend this list of examples.

Therefore the physical community always follows the developments in hardware and software with great interest. Developments in this area are so fast that if a particular type of machine were presented here as being today's state of the art, this statement would be outdated by the time the book is on the shelf. We therefore restrict ourselves here to a short account of some general principles of computer architecture and implications for software technology. The two main principles are pipelining and parallelism. Both concepts were developed a few decades ago, but pipelining became widespread in supercomputers from around 1980 and has found its way into most workstations, whereas parallelism has remained more restricted to the research community and to more expensive machines. The reason for this is that it is easier to modify algorithms to make them suitable for pipelining than it is for parallelism. Recently, the dual-core processor has started to find its way into consumer PCs.

Conventional computers are built according to the *Von Neumann* architecture, shown in the figure below:



There is one processor, the CPU (central processing unit), which communicates with the internal memory and with the I/O devices such as keyboards, screens, printers, tape streamers and disks. Therefore every piece of data to be manipulated must pass through the processor which can perform elementary operations at a fixed rate of one operation per *clock cycle*. The clock cycle time determines the maximal speed at which the computer can operate. This limit on the performance speed with this type of architecture is called the *Von Neumann bottleneck*.

In the next two sections the two methods for overcoming the Von Neumann bottleneck, pipelining and parallelism, are discussed, and in the last section we present a parallel molecular dynamics algorithm in some detail.

16.2 Pipelining

16.2.1 Architectural aspects

Pipelining or *vector processing* is closely related to a pipeline arrangement in a production line in a factory. Consider the addition of two floating point numbers, 0.92×10^4, and 0.93×10^3, in a computer. We shall assume that the numbers in our computer are represented according to the decimal (base 10) system – in reality mostly a base 2 (binary) or 16 (hexadecimal) system is used. The exponents (powers of 10) are always chosen such that the mantissas (0.92 and 0.93) lie between 0.1 and 1.0.

The computer will first change the representation of one of the two numbers so that it has the same exponent as the other, in our case $0.93 \times 10^3 \rightarrow 0.093 \times 10^4$, then the two mantissas (0.92 and 0.093) are added to give the result 1.013×10^4 and then the representation of this number will be changed into one in which the mantissa has a value between 0.1 and 1.0: $1.013 \times 10^4 \rightarrow 0.1013 \times 10^5$. All in all, a number of steps must be carried out in the processor in order to perform this floating point operation:

Load the two numbers to be added from memory;
Compare exponents;
Line exponents up;
Add mantissas;

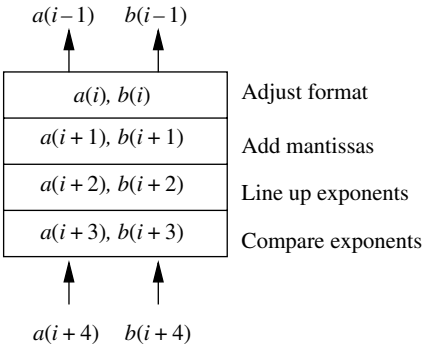


Figure 16.1. A pipeline for adding vectors.

Shift exponent such that the mantissa lies between 0.1 and 1.0;
Write result to memory.

Disregarding the load from and write to memory, we still have four steps to carry out for the addition. Each of these steps requires at least one clock cycle, and a conventional processor has to wait until the last step has been completed before it can accept a new command.

A pipeline processor, however, can perform the different operations needed to add two floating point numbers at the same time (in parallel). This is of no use when only two numbers are added, as this calculation must be completed before starting execution of the next statement. However, if we have a sequence of similar operations to be carried out, like in the addition of two vectors:

FOR $i = 1$ TO N DO
    $c[i] = a[i] + b[i]$;
END FOR

then it is possible to have the processor comparing the exponents of $a[i + 3]$ and $b[i + 3]$, lining up the exponents of $a[i + 2]$ and $b[i + 2]$, adding the mantissas of $a[i + 1]$ and $b[i + 1]$ and putting $a[i]$ and $b[i]$ into the right format *simultaneously*. Of course this process acts at full speed only after $a[4]$ and $b[4]$ have been loaded into the processor and only until $a[N]$ and $b[N]$ have entered it. Starting up and emptying the pipeline therefore represent a small overhead. Figure 16.1 shows how the process works and also renders the analogy with the pipeline obvious.

In the course of this pipeline process, one addition is carried out at each clock cycle. We call an addition or multiplication of real numbers a *floating point operation* (FLOP). We see that the pipeline arrangement makes it possible to perform one FLOP (FLOP) per clock cycle. Multiplication and division require many more than four clock cycles in a conventional processor, and if each of the steps involved in the multiplication or division can be executed concurrently in a pipeline

processor, the speed-up can be much higher than for addition. Often, pipeline processors are able to run a pipeline for addition and multiplication simultaneously, and the maximum obtainable speed, measured in floating point operations per second, is then increased by a factor of two. A pipeline processor with a clock time of 5 ns can therefore achieve a peak performance of $2/(5 \times 10^{-9}) = 400$ MFLOPs per second ($1\,\text{MFLOP} = 10^6\,\text{FLOPS}$).

In practice, a pipeline processor will not reach its peak performance for various reasons. First, the overheads at the beginning and the end of the pipeline slightly decrease the speed, as we have seen. More importantly, a typical program does not exclusively contain simultaneous additions and multiplications of pairs of large vectors. Finally, for various operations, pipeline machines suffer from slow memory access. This is a result of the fact that memory chips work at a substantially slower rate than the processor, simply because faster memory is too expensive to include in the computer. The typical difference in speed of the two is a factor of four (or more). This means that the pipeline cannot feed itself with data at the same rate at which it operates, nor can it get rid of its data as fast as they are produced. In so-called *vector processors*, the data are read into a fast processor memory, the so-called *vector registers*, from which they can be fed into the pipelines at a processor clock-cycle rate. This still leaves the problem of feeding the vector registers at a fast enough rate, but if the same vectors are used more than once, a significant increase increase in performance is realised.

Because of this problem, which may cause the performance for many operations to be reduced by a factor of four, various solutions have been developed. First of all, the memory is often organised into four or more *memory banks*. Suppose that we have four banks, and that each bank can be accessed only once in four processor clock cycles (i.e. the memory access time is four times as slow as the processor clock cycle), but after accessing bank number one, bank number two (or three or four) can be accessed immediately at the next clock cycle. Therefore, the banks are cycled through in succession (see Figure 16.2). For this to be possible, vectors are distributed over the memory banks such that for four banks, the first bank contains the elements 1, 5, 9, 13 etc., the second one the elements 2, 6, 10, 14 . . . and so on. Cycling through the memory banks enables the processor to fetch a vector from memory at a rate of one element per clock cycle. In the case of the vector-addition code above, this is not enough as two numbers are required per clock cycle. Also, if we want to carry out operations on the vector elements 1, 5, 9 . . . of a vector, the memory access time increases by a factor of four. Such problems are called *memory bank conflicts*.

Another device for solving the slow memory problem is based on the observation that a program will often manipulate a restricted amount of data more than once. Therefore a relatively small but fast memory, the so-called *cache memory*, is included. This memory acts as a buffer between the main memory and the processor, and it contains the data which have been accessed most recently by the processor.

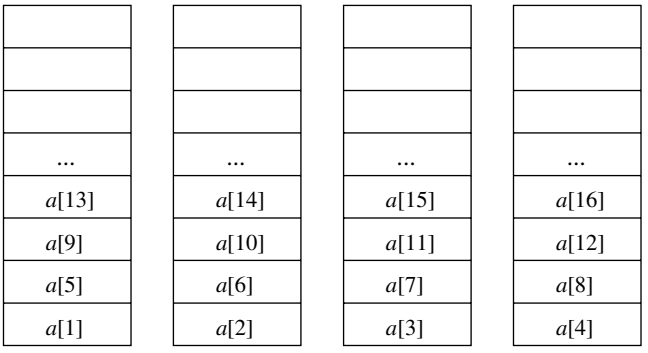


Figure 16.2. Distribution of one-dimensional array over four memory banks.

Therefore, if a first pipeline using vectors a, b and c is executed, it will cause a rather large overhead because these arrays have to be loaded from main memory into the cache. However, a subsequent pipeline using (a subset of) a, b and c will run much faster because these vectors are still stored in the cache. Obviously the cache paradigm is based on statistical considerations, and for some programs it may not improve the performance at all because most data are directly fetched from main memory (*cache trashing*). Sometimes, the cache divided into several levels of increasing speed and cost, but decreasing size. Level-1 cache is built into the processor; level-2 cache is either part of the processor, or external.

16.2.2 Implications for programming

For a pipeline process to be possible, it should be allowed to run without interruption. If the processor receives commands to check whether elements of the vector being processed are zero, or to check whether the loop must be interrupted, it cannot continue the pipeline and performance drops dramatically. Pipelines usually work in vector processing mode, in which the processor receives a command such as 'calculate the scalar product' or 'add two vectors' with as operands the memory locations (addresses) of the first elements of the vectors, the length of the pipeline and the address of the output value (scalar or vector). How can we tell the processor to start a pipeline rather than operate in conventional mode? This is usually done by the compiler which recognises the parts of our program which can be pipelined. In the language Fortran 90 the statement

```
c = a + b,
```

for one-dimensional arrays `a`, `b` and `c` is equivalent to the vector addition considered above. This can easily be recognised by a compiler as a statement which can be executed in pipeline mode.

If we want our program to be pipelined efficiently, there are a few dos and don'ts which should be kept in mind.

- Avoid conditional statements ('if-statements') within loops. Also avoid subroutine or function calls within loops.
- In the case of nested loops whose order can be interchanged, the longest loop should always be the interior loop.
- Use standard, preferably machine-optimised software to perform standard tasks.
- If possible, use indexed loops ('do-loops') rather than conditional loops ('while-loops').

A few remarks are in order. In the third item, use of standard software for standard tasks is recommended. There exists a standard definition, called LAPACK, of linear algebra routines. Often, vendors provide machine-optimised versions of these libraries; these should always be used if possible.

If a function call or IF-statement cannot be avoided, consider splitting the loop into two loops: one in which the function calls or if-statements occur, and another in which the vectorisable work is done. Some computers contain a *mask-register* which contains some of the bits which are relevant to a certain condition (e.g. the sign bit for the condition $a[i] > 0$). This mask register can be used by the processor to include the condition in the pipeline process. Finally, compiling with the appropiate optimisation options will, for some or perhaps most of the above recommendations, automatically implement the necessary improvements in the executable code.

16.3 Parallelism

16.3.1 Parallel architectures

Parallel computers achieve increase in performance by using multiple processors, which are connected together in some kind of network. There is a large variety of possible arrangements of processors and of memory segments over the computer system. Several architecture classifications exist. The most famous classification is that of Flynn who distinguishes the following four types of architectures [1, 2]:

- SISD: Single Instruction Single Data stream computers;
- MISD: Multiple Instruction Single Data stream computers;
- SIMD: Single Instruction Multiple Data stream computers;
- MIMD: Multiple Instruction Multiple Data stream computers.

The single/multiple instruction refers to the number of instructions that can be performed concurrently on the machine, and the single/multiple data refers to whether the machine can process one or more data streams at the same time.

We have already described the SISD type: this is the Von Neumann architecture, in which there is one processor which can process a single data stream: the sequence of data which is fetched from or written to memory by the processor. The second type, MISD is not used in practice but it features in the list for the sake of completeness. The two last types are important for parallel machines. SIMD machines consist of arrays of processor elements which have functional units controlled by a single control unit: this unit sends a message to all processors which then all carry out the same operation on the data stream accessed by them. MIMD machines consist of an array of processors which can run entire programs independently, and these programs may be different for each processor. In very large-scale scientific problems, most of the work often consists of repeating the same operation over and over. SIMD architectures are suitable for such problems, as the processor elements can be made faster than the full processors in a MIMD machine at the same cost. The latter obviously offer greater flexibility and can be used as multi-user machines, which is impossible for SIMD computers.

Another classification of parallel machines is based on the structure of the memory. We have *distributed memory* and *shared memory* architectures. In the latter, all processors can all access the same memory area using some communication network (sometimes they can also communicate the contents of vector registers or cache to the other processors). In distributed systems on the other hand, each processor has its own local memory, and processors can interact with each other (for example to exchange data from their local memories) via the communication network. Some machines can operate in both modes. Shared memory architectures are easier to program than distributed memory computers as there is only a single address space. Modifying a conventional program for a shared memory machine is rather easy. However, memory access is often a bottleneck with these machines. This will be clear when you realise that memory bank conflicts are much more likely to occur when 10 processors are trying to access the memory instead of a single one. Distributed memory systems are more difficult to program, but they offer more potential if they are programmed adequately. The shared memory model is adopted in several supercomputers and parallel workstations containing of the order of 10 powerful vector processors. However, the most powerful machines at present and for the future seem to be of the distributed memory type, or a mixture of both.

In parallel machines, the *nodes*, consisting of a processor, perhaps with local (distributed) memory, must be connected such that data communication can proceed fast, but keeping an eye on the overall machine cost. We shall mention some of these configurations very briefly. The most versatile option would be to connect each processor to each of the others, but this is far too expensive for realistic designs. Therefore, alternatives have been developed which are either tailored to particular problems, or offer flexibility through the possibility of making different connections

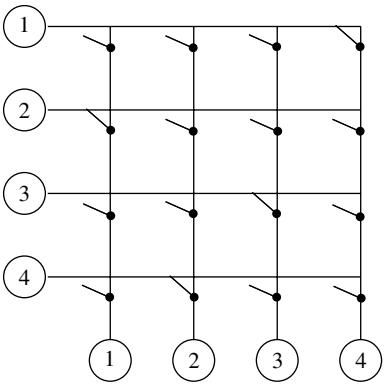


Figure 16.3. A crossbar switch connecting four processors. Each processor has two I/O ports and these are connected together via the crossbar switch. Denoting by (i,j) a connection from i to j, the switch configuration shown is (1,4), (2,1), (3,3), (4,2).

using *switches*. An example of an architecture with switches is a *crossbar switch*, shown in Figure 16.3. On each row and on each column, only a single switch may be connected – in this way binary connections are possible for any pair-partitioning of the set of processors.

Another way to interconnect processors is to link them together in a grid or chain, where each processor can communicate with its nearest neighbours on the grid or chain. Usually, the grids and chains are periodic, so that they have the topology of a ring or a torus. Rings and two- and three-dimensional tori have the advantage that they are *scalable*: this means that with increasing budget, more processors can be purchased and the machine performance increased accordingly. Furthermore, some problems in physics and engineering map naturally onto these topologies, such as a two- or three-dimensional lattice field theory with periodic boundary conditions which maps naturally onto a torus of the same dimension.

The problem of sending data from one processor to the other in the most efficient way is called *routing*. In older machines, this data-traffic, called *message passing*, was often a bottleneck for overall performance, but today this is less severe (although still a major concern).

Another type of network is the *binary hypercube*. This consists of 2^d nodes, which are labelled sequentially. If the labels of two nodes differ by only one bit, they are connected. The hypercube is shown for $d = 1$ to 4 in Figure 16.4. This network has many more links than the ones which have essentially a two-dimensional layout, and the substructures of the hypercube include multi-dimensional grids and trees.

In connection with parallel processors, *Amdahl's law* is often mentioned. This law imposes an upper limit on the increase in performance of a program by running

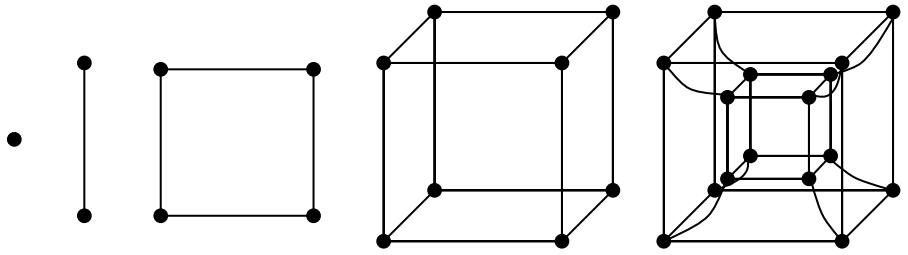


Figure 16.4. Hypercubes of order 0 (single dot) to order 4 (two cubes, one inside the other).

it on a parallel machine. Each job can partly be done in parallel, but there will always be some parts that have to be carried out sequentially. Suppose that on a sequential computer, a fraction p_{seq} of the total effort consists of jobs which are intrinsically sequential and that a fraction $p_{\text{par}} = 1 - p_{\text{seq}}$ can in principle be carried out in parallel. If we ran the program on a parallel machine and distributed the parallel parts of the code over P processors, the total run time would decrease by a factor

$$\frac{t_{\text{seq}}}{t_{\text{par}}} = \frac{p_{\text{seq}} + p_{\text{par}}}{p_{\text{seq}} + p_{\text{par}}/P} = \frac{1}{p_{\text{seq}} - (1 - p_{\text{seq}})/P} \tag{16.1}$$

(t_{seq} and t_{par} are the total run times on the sequential and parallel machine respectively). The *processor efficiency* is defined by

$$\text{P.E.} = \frac{1}{Pp_{\text{seq}} + (1 - p_{\text{seq}})}. \tag{16.2}$$

The processor efficiency is equal to 1 if all the work can be evenly distributed over the P processors.

Even if we could make P arbitrarily large, the maximum decrease in runtime can never exceed $1/p_{\text{seq}}$ – this is Amdahl's law – and the processor efficiency will decrease as $1/P$. We see that for larger P, increase of performance decays to zero. Amdahl's law should not discourage us too much, however, since if we have a larger machine, we usually tackle larger problems, and as a rule, the parallelisable parts scale with a higher power of the problem size than the sequential parts. Therefore, for really challenging problems, a processor efficiency of more than 90% can often be achieved.

16.3.2 Programming implications

Writing programs which exploit the potential of parallel computers is different from vectorising, as there are many more operations and procedures that can be parallelised than can be vectorised. Moreover, we might want to tailor our program to the particular network topology of the machine we are working on. Ideally,

compilers would take over this job from us, but unfortunately we are still far from this utopia. There exist various programming paradigms which are in use for parallel architectures. First of all, we have *data-parallel* programming versus *message-passing* programming. The first is the natural option for shared memory systems, although in principle it is not restricted to this type of architecture.

In data-parallel programming, all the data are declared in a single program, and at run-time these data must be allocated either in the shared memory or suitably distributed over the local memories, in which case the compiler should organise this process. For example, if we want to manipulate a vector $a[N]$, we declare the full vector in our program, and this is either allocated as a vector in the shared memory or it is chopped into segments which are allocated to the local memory of the processors involved. The message-passing model, however, is suitable only for distributed memory machines. Each processor runs a program (they are either all the same or all different, according to whether we are dealing with a SIMD or MIMD machine) and the data are allocated locally by that program. This means that if the program starts with a declaration of a real variable a, this variable is allocated at each node – together these variables may form a vector.

As an example we consider the problem in which we declare the vector $a[N]$ and initialise this to $a[i] = i, i = 1, \ldots, N$. Then we calculate:

FOR $i = 1$ TO N DO
  $a[i] = a[i] + a[((i - 1)$ MOD N$) + 1]$;
END DO

In Fortran 90, in the data-parallel model, this would read:

```
1  INTEGER, PARAMETER :: N=100          ! Declaration of
                                        ! array size
2  INTEGER, DIMENSION(N) :: A, ARight ! Declare arrays

3  DO I=1, N                            ! Initialise A

4    A(I)=I

5  END DO
6  ARight=CSHIFT(A, SHIFT=-1, DIM = 1)! Circular shift
                                        ! of A
7  A = A + ARight                       ! Add A and
                                        ! ARight
                                        ! result stored
                                        ! in B
```

In this program, the full vectors A and ARight are declared – if we are dealing with a distributed memory machine, it is up to the compiler to distribute these over the nodes. The command CSHIFT returns the vector *A*, left-shifted in a cyclical fashion over one position. In the last statement, the result is calculated in vector notation (Fortran 90 allows for such vector commands). This program would run equally well on a conventional SISD machine – the compiler must find out how it distributes the data over the memory and how the CSHIFT is carried out.

In a message-passing model, the program looks quite different:

```
1  INTEGER, PARAMETER :: N =100          ! Declaration of
                                         ! array size
2  INTEGER :: VecElem, RVecElem          ! Elements of
                                         ! the vector
3  INTEGER :: MyNode, RightNode, &       ! Variables to
              LeftNode                   ! contain node
                                         ! addresses
4  MyNode = whoami()                     ! Determine node
                                         ! address
5  VecElem = MyNode                      ! Initialise
                                         ! vector element
6  RightNode = MOD(MyNode+1, N)+1        ! Address of
                                         ! right neighbour
7  LeftNode = MOD(Mynode-1+N, N)+1       ! Address of left
                                         ! neighbour
8  CALL send&get(VecElem, LeftNode, &    ! Circular
                 RVecElem, RightNode)    ! left-shift
9  VecElem = VecElem + RVecElem          ! Calculate sum
```

This program is more difficult to understand. In the program it is assumed that there are at least 100 free processors. They are numbered 1 through 100. The function whoami() returns the number of the processor – at each processor, MyNode will have a different value. The variable VecElem is an INTEGER which is allocated locally on each processor: the same name, VecElem is used to denote a collection of different numbers. Referring to this variable on processor 11 gives in general a different result than on processor 37. The right and left neighbours are calculated in statements 6 and 7. Statement 8 contains the actual message passing: VecElem, the element stored at the present processor, is sent to its left neighbour and the element of the right hand neighbour, RVecElem, is received at the present processor. In statement 9 this is then added to VecElem. A popular system for parallel communication in the message-passing paradigm is the *message-passing system* MPI-2, which is highly portable and offers extensive functionality.

Another classification of programming paradigms is connected with the organisation of the program. A simple but efficient way of parallelisation is the *master–slave*

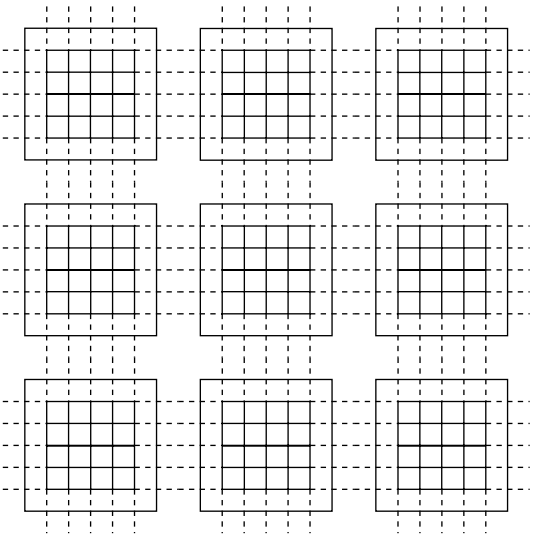


Figure 16.5. The Ising model partitioned over the nodes of a two-dimensional torus. The nodes are indicated as the heavy squares and on each node part of the lattice is shown. The dashed lines represent the couplings between spins residing on neighbouring nodes.

or *farmer–worker* model. In this model, one processor is the farmer (master) who tells the workers (slaves) to do a particular job. The jobs are carried out by the workers independently, and they report the results to the farmer, not to each other. An example is the solution of the Schrödinger equation for a periodic solid. As we have seen in Chapter 6, Bloch's theorem can be used, which tells us that the Schrödinger equation for each particular Bloch vector $\mathbf{k}$ in the Brillouin zone can be carried out independently. So, if the effective one-electron potential is known, the farmer will tell the workers to diagonalise the Hamiltonian for a set of $\mathbf{k}$-vectors allocated previously to the worker nodes. If the workers have finished the diagonalisation, they send the result back to the farmer who will calculate the charge density for example. Parts of the latter calculation could also be distributed over the workers.

Another model is the peer-to-peer model, in which the nodes are equivalent; they exchange data without one processor telling the others what to do. An example is an MC program for the Ising lattice which is partitioned over a two-dimensional torus. Each node performs the Metropolis or heat-bath algorithm for the part of the lattice which has been allocated to it, but it needs the neighbouring spins residing on other nodes. The idea is represented in Figure 16.5.

Finally, in the macro-pipelining model, each node performs a subtask of a large job. It yields processed data which are then fed into the next node for further processing. The nodes thus act in a pipelined mode, hence the name macro-pipelining, as opposed to micro-pipelining which takes place within a (pipeline-) processor. An example is the processing of a huge amount of time-series consisting

of a set of N real values. The aim is to filter the data according to some scheme defined in terms of the Fourier components. In that case, the first node would Fourier-transform the data, the second one would operate on the Fourier coefficients (this is the actual filter) and a third one would Fourier-transform the results back to real time; the three parts are arranged in a pipeline.

16.4 Parallel algorithms for molecular dynamics

In Chapter 8 we discussed the molecular dynamics (MD) method in some detail. In the standard microcanonical MD algorithm, Newton's equations of motion are solved in order to predict the evolution in time of an isolated classical many-particle system. From the trajectory of the system in phase space we can determine expectation values of physical quantities such as temperature, pressure, and the pair correlation function.

We consider the MD simulation of argon in the liquid phase, which was discussed rather extensively in Chapter 8. The total interaction energy is a sum of the Lennard–Jones potentials between all particle pairs. The force on a particular particle is therefore the sum of the Lennard–Jones forces between this and all the other particles in the system. The evaluation of these forces is the most time-consuming step of the simulation, as this scales as the square of the number N of particles in the system. It therefore seems useful to parallelise this part of the simulation.

There exist essentially two methods for doing this. The first method is based on a spatial decomposition of the system volume: the system is divided up into cells, and the particles in each cell are allocated to one node. The nodes must have the same topology as the system cells, in the sense that neighbouring cells should be allocated to neighbouring nodes whenever possible. The particles interact with the other particles in their own cell, and with the particles in neighbouring cells. Interactions between particles in non-neighbouring cells are neglected. The smallest diameter of a cell must therefore be large enough to justify this approximation: in other words, the Lennard–Jones interaction must be small enough for this distance and beyond. A problem is that particles may tend to cluster in some regions in phase space, in particular when there is phase separation or when the system is close to a second order phase transition. Then some nodes may contain many more particles than others. In that case the nodes with fewer particles remain inactive for a large part of the time. This is a general problem in parallelism, called *load balancing*. Moreover, particles move and may leave the cell they are in, and therefore the allocation of the particles to the cells must be updated from time to time (similar to the procedure in Verlet's neighbour list: see Section 8.2).

The simplest version of this algorithm is realised by dividing up the system along one dimension. In this way, one obtains slices of the system, the thickness of which

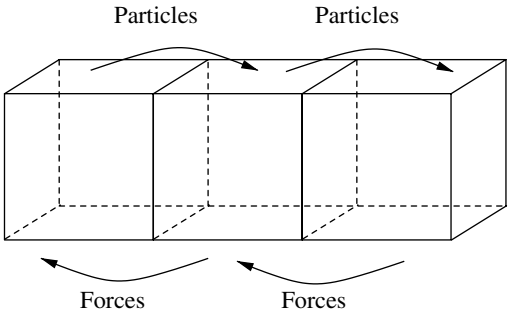


Figure 16.6. The parallel molecular dynamics procedure. At each time step, the particle positions are sent to the right neighbours. There, the forces on the local particles, and on the particles just received from the left box, are calculated. These forces are then sent back to the left slice so that they can be added to the total force calculated there. Now the particles can be moved according to the Verlet algorithm.

should exceed the cut-off of the Lennard–Jones force. The force evaluation is then implemented according to the following scheme.

```
Send particles in slice to the right neighbour;
Receive guest particles from left neighbour;
FOR all particle pairs in present slice DO
    Calculate forces for particles in present slice due to
            particles in this slice and store their values;
END FOR;
FOR all particles in present slice DO
    FOR all particles received from left neighbour DO
        Calculate forces between 'resident' and 'guest' particle;
        Store force on resident particle in resident force array;
        Store force on 'guest' particle in a sending array
    END FOR;
END FOR;
Send forces on guest particles to left neighbour;
Receive forces on resident particles from right neighbour;
Add these last forces to total forces on resident particles;
Move particles according to Verlet algorithm.
```

The procedure is represented in Figure 16.6.

To evaluate this method, we give results for the performance of an MD simulation for a rectangular box of size $NL \times L \times L$. This was divided up into N cubic $L \times L \times L$ boxes, each of which was allocated to a different processor. In total $N \times 2048$ particles were present in the system, where N is the number of processors.

The density was 1.0 and the temperature was 0.8 (in reduced units). Running this program for 1000 time steps on a SGI Altix MIMD system took 20 ± 1 seconds per step using 4, 8 and 16 processors.

A second approach focuses on the evaluation of the double sum in the force calculation. The p nodes available are connected in a ring topology: each node is connected to a left and a right neighbour, and the leftmost node is connected to the rightmost node and vice versa. The particles are again divided over the nodes, but not in a way depending on their positions in the physical system: of the N particles, the first N/p are allocated to node number 1, the next N/p to node number 2 and so on (we suppose that N/p is an integer for simplicity). Each node therefore contains the positions and momenta of the particles allocated to it (we shall use the leap-frog algorithm for integrating the equations of motion) – these are called the *local particles*. In addition to these data, each node contains memory to store the forces acting on its local particles, and sufficient memory to contain positions and momenta of 'travelling particles', which are to be sent by the other nodes. The number of travelling particles at a node equals the number of local particles, and therefore we need to keep free memory available for $6N/p$ real numbers in order to be able to receive these travelling particle data.

In the first step of the force calculation, all the interactions of local particles with other local particles are calculated, as described in Chapter 8. Then each node copies its local positions and momenta to those of the travelling particles. Each node then sends the positions and momenta of its travelling particles to the left, and it receives the data sent by its right neighbour into its travelling particle memory area. At this point, node 1, which still contains the momenta and positions of its local particles in its local particle memory area, contains the positions and momenta of the local particles of node 2 in its travelling memory area. The travelling particles of node 1 have been sent to node p.

Now the contributions to the forces acting on the local particles of each node resulting from the interactions with the travelling particles are calculated and added to the forces. Then the travelling particles are again sent to the left neighbour and received from the right neighbour (modulo PBC), and the calculation proceeds in the same fashion. When each segment of travelling particles has visited all the nodes in this way, all the forces have been calculated, and the new local positions and momenta are calculated according to the leap-frog scheme in each processor. Then the procedure starts over again, and so on. The motion of the particles over the nodes in the force calculation is depicted in Figure 16.7. This algorithm is called the *systolic algorithm* [3], because of the similarity between the intermittent data-traffic and the heart beat.

We now give the systolic algorithm in schematic from, using a data-parallel programming mode. This means that at each node, memory is allocated for the local particles, forces and travelling particles, but the data contained in this memory are

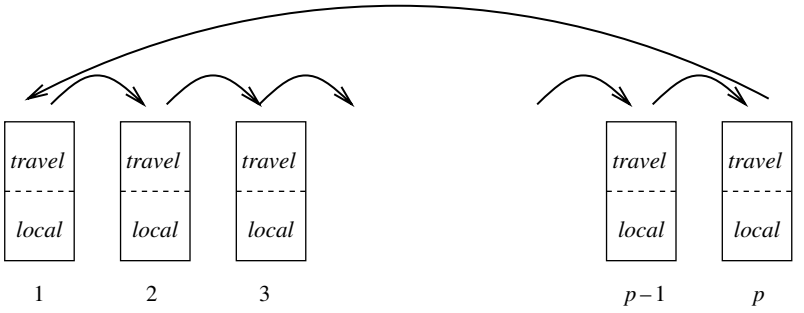


Figure 16.7. The systolic algorithm for calculating the forces in a many-particle system with pair-wise interactions.

different for each node. It is assumed that the local particles are initialised correctly at the beginning – in practice this is done by a so-called *host processor*, which is either one of the nodes, or a so-called front-end computer, which usually executes the sequential parts of a program.

```
ROUTINE NodeStep
    REPEAT
        Calculate forces on local particles;
        Perform leap-frog step for local particles;
    UNTIL Program stops;
END ROUTINE Node Step;
ROUTINE Calculate forces on local particles
    Calculate contributions from local particles;
    Copy local particles to travelling particles;
    DO K = 1, P − 1
        Send travelling particles to the left (modulo PBC);
        Receive data from right neighbour and load into
                memory used by travelling particles;
        Calculate contributions to local forces from travelling particles;
    END DO
END ROUTINE Calculate forces on local particles;
```

The systolic algorithm does not suffer from load-balance problems, as the work to be executed at each node is the same. Only if the number of particles N is not equal to an integer times the number of nodes does the last processor contain fewer particles than the remaining ones, so that a slight load-imbalance arises.

In the form described above, the systolic algorithm does not exploit the fact that for every pair ij, the force exerted by particle j on particle i is opposite to the force exerted by i on j, so that these two force contributions need only one calculation. However, a modification exists which removes this disadvantage [4]. A drawback

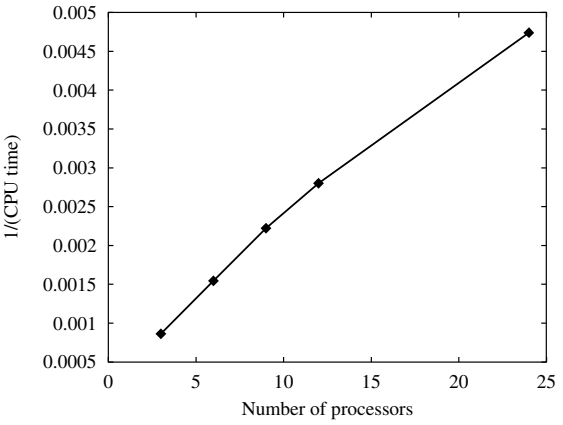


Figure 16.8. Execution speed (given as the inverse of the execution time in seconds) of the systolic MD algorithm for 864 particles executing 750 steps on a Transtech Paramid parallel computer.

of the systolic algorithm is the fact that the interactions between *all* particle pairs are taken into account in the force calculation, whereas neglecting contributions from pairs with a large separation, in the case of rapidly decaying forces, can reduce the number of calculations considerably without affecting the accuracy noticeably. Of course we can leave out these calculations in the systolic algorithm as well, but this does not reduce the total number of steps (although some steps will be carried out faster) and it requires the use of IF-statements within loops, which may degrade performance of nodes with pipelining facilities (see Section 16.2.2). Therefore, the efficiency gain by parallelising the force calculation (and the integration of the equation of motion) may not be dramatic for rapidly decaying interactions.

In Figure 16.8, the execution speed is shown as a function of the number of processors for an 864 Lennard–Jones system, integrating 750 time steps. A parallel program is called *scalable* if the execution speed is proportional to the number of processors used. We see that for larger numbers of processors, the scalability gets worse as a result of the increasing amount of interprocessor communication. This is noticeable here because the runs were performed on a rather old machine: a Transtech Paramid. On modern machines, the interprocessor communication is so fast that this algorithm would be seen to be almost perfectly scalable.

References

[1] M. J. Flynn, 'Very high speed computing systems,' *Proc. IEEE*, **54** (1966), 1901–9.
[2] M. J. Flynn, 'Some computer architectures and their effectiveness,' *IEEE Trans. Comp.*, **C-21** (1972), 948–60.
[3] W. D. Hillis and J. Barnes, 'Programming a highly parallel computer,' *Nature*, **326** (1987), 27–30.
[4] A. J. van der Steen, ed., *Aspects of Computational Science: A Textbook on High-Performance Computing*. Den Haag, National Computing Facilities Foundation, 1995.

Appendix A

Numerical methods

A1 About numerical methods

In computational physics, many mathematical operations have to be carried out numerically. Techniques for doing this are studied in numerical analysis. For a large variety of numerical problems, commercial or public domain software packages are available, and on the whole these are recommended in preference to developing all the numerical software oneself, not only because this saves time but also because available routines are often coded by professionals and hence are hard to surpass in generality and efficiency. To avoid eventual problems with using existing numerical software, it is useful to have some background in numerical analysis. Moreover, knowledge of this field enables you to write codes for special problems for which no routine is available. This chapter reviews some numerical techniques which are often used in computational physics.

There are several ways of obtaining 'canned' routines. Commercially available libraries (such as NAG, IMSL) are of very high quality, but often the source code is not available, which might prevent your software from being portable. However, several libraries, such as the ones quoted, are available at many institutes and companies for various types of computers ('platforms'), so that in practice this restriction is not so severe.

Via the internet, it is possible to obtain a wide variety of public domain routines; a particularly useful site is `/www.netlib.org/`. Most often these are provided in source code. Another cheap way of obtaining routines is by purchasing a book on numerical algorithms containing listings of source codes and, preferably, a CD (or an internet address) with these sources. A well-known book is *Numerical Recipes* by Press *et al.* [1]. This is extremely useful: it contains very readable introductions to various subjects in numerical mathematics and describes many algorithms in detail. It explains the pros and cons of the methods and, most importantly, it explains what can go wrong and why. Source codes are provided on diskette or CD in C, Fortran and Pascal, although in some cases people have found it better to use routines from NAG or Netlib. Apart from *Numerical Recipes* there are many excellent books in

the field of numerical mathematics [2–6] and the interested reader is advised to consult these.

This appendix serves as a refresher to those readers who have some knowledge of the subject. Novice readers may catch an idea of the methods and can look up the details in a specialised book. The choice of problems discussed in this appendix is somewhat biased: although several methods described here are not used in the rest of the book, the emphasis is on those that are.

A2 Iterative procedures for special functions

Physics abounds with special functions: functions which satisfy classes of differential equations or given by some other prescription, and which are usually more complicated than simple sines, cosines or exponentials. Often we have an iterative prescription for determining these functions. Such is the case for the solutions to the radial Schrödinger equation for a free particle:

$$\left[-\frac{1}{2}\frac{\mathrm{d}^2}{\mathrm{d}r^2}+\frac{l(l+1)}{2r^2}\right][rR_l(r)] = E[rR_l(r)], \tag{A.1}$$

where the units are chosen such that $\hbar^2/m \equiv 1$ (it is always useful to choose such natural units to avoid cumbersome exponents). The solutions R_l of (A.1) are known as the *spherical Bessel functions* $j_l(kr)$ and $n_l(kr)$, $k = \sqrt{2E}$. The j_l are regular for $r = 0$ and the n_l are irregular (singular). For $l = 0, 1$, the spherical Bessel functions are given by:

$$\begin{aligned} j_0(x) &= \frac{\sin(x)}{x}; & n_0(x) &= -\frac{\cos(x)}{x}; \\ j_1(x) &= \frac{\sin(x)}{x^2} - \frac{\cos(x)}{x}; & n_1(x) &= -\frac{\cos(x)}{x^2} - \frac{\sin(x)}{x}. \end{aligned} \tag{A.2}$$

For higher l, we can find the functions by:

$$s_{l+1}(x) + s_{l-1}(x) = \frac{2l+1}{x}s_l(x) \tag{A.3}$$

where s_l is either j_l or n_l. Equation (A.3) gives us a procedure for determining these functions numerically. Knowing for example j_0 and j_1, Eq. (A.3) determines j_2 and so on. A three-point recursion relation has two independent solutions, and one of these may grow strongly with l. If the solution we are after damps out and the other one grows with l, the solution is unstable with respect to errors that will always sneak into the solution owing to the fact that numbers are represented with finite precision in the computer. It turns out that j_l damps out rapidly with increasing l, so that it is sensitive to errors of this kind, especially when l is significantly larger than x. One can avoid such inaccuracies by performing the recursion downwards

instead of upwards. This is done by starting at a value l_{top} lying significantly higher than the one we want the Bessel function for. We put $s_{l_{\text{top}}+1}$ equal to zero and $s_{l_{\text{top}}}$ equal to some small value δ. Then, the recursion procedure is carried out downward until one arrives at the desired l. Notice that the normalisation of the solution is arbitrary (since it is determined by the arbitrary small number δ). To normalise the solution, we continue recursion down to $l = 0$ and then use the fact that $(j_0 - xj_1)\cos x + xj_0 \sin x = 1$ which determines the normalisation constant.

A problem is of course with how large an l_{top} one should start. In many cases this can be read off from the asymptotic expression for the functions to be determined.

A3 Finding the root of a function

An important problem in numerical analysis is that of finding the root of a one-dimensional, real function f:

$$f(x) = 0. \tag{A.4}$$

If this problem cannot be solved analytically, numerical alternatives have to be used, and here we shall describe three of these, the regula falsi, the secant method and the Newton–Raphson method. Of course, the function f may have more than one or perhaps no root in the interval under consideration; some knowledge of f is therefore necessary for the algorithms to arrive at the desired root. If a continuous function has opposite signs at two points x_1 and x_2, it must have at least one root between these points. If we have two points x_1 and x_2 where our function has equal signs, we can extend the interval beyond the point where the absolute value of f is smallest until the resulting interval brackets a root. Also, if the number of roots between x_1 and x_2 is even, the function has the same sign on both points; if we suspect roots to lie within this interval, we split the interval up into a number of smaller subintervals and check all of them for sign changes. We suppose in the following that the user roughly knows the location of the root (it might be bracketed, for example), but that its precise location is required.

The efficiency of a root-finding method is usually expressed in terms of a convergence exponent y for the error in the location of the root. Since root-finding methods are iterative, the expected error, δ_n, at some step in the iteration can be expressed in terms of the error at the previous step, δ_{n-1}:

$$\delta_n = \text{Const.} \times |\delta_{n-1}|^y. \tag{A.5}$$

The *regula falsi*, or *false position* method, starts with evaluating the function at two points, x_1 and x_2, lying on either side of a single root, with $x_1 < x_2$. Therefore the signs of $f(x_1) \equiv f_1$ and $f(x_2) \equiv f_2$ are opposite. A first guess for the root is

found by interpolating f linearly between x_1 and x_2, resulting in a location

$$x_3 = \frac{x_2 f_1 - x_1 f_2}{f_1 - f_2}. \tag{A.6}$$

If $f_3 = f(x_3)$ and f_1 have opposite signs, we can be sure that the root lies between x_1 and x_3 and the procedure is repeated for the interval $[x_1, x_3]$, else the root lies in the interval $[x_3, x_2]$ and this is used in the next step. The procedure stops when either the size of the interval lies below a prescribed value, or the absolute value of the function is small enough.

Instead of checking which of the intervals, $[x_1, x_3]$ or $[x_3, x_2]$ contains the root, we might simply use the two x-values calculated most recently to predict the next one. In that case we use an equation similar to (A.6):

$$x_{n+1} = \frac{x_n f_{n-1} - x_{n-1} f_n}{f_{n-1} - f_n}. \tag{A.7}$$

Here, x_n is the value of x obtained at the nth step, so the points x_n and x_{n-1} do not necessarily enclose the root. This method, called the *secant method*, is slightly more efficient than the regula falsi; *regula falsi method* it has a convergence exponent equal to the golden mean 1.618 . . . . If the function has a very irregular behaviour, these methods become less efficient and it is more favourable in that case to take at each step the middle of the current bracket interval as the new point, and then check whether the root lies in the left or in the right half of the current interval. This method is called the bisection method. As the size of the intervals is halved at each step, the exponent y takes on the value 1. For more elaborate and fault-proof methods, see Ref. [1].

For the Newton–Raphson method, the derivative of the function must be calculated, which is not always possible. Again, a prediction based on a linear approximation of the function f is made. Starting with a point x_0 lying close to the root, the values of $f(x_0) \equiv f_0$ and of its derivative $f'(x_0) \equiv f'_0$ suffice to make a linear interpolation for f, resulting in a guess x_1 for the location of the root:

$$x_1 = x_0 - \frac{f_0}{f'_0}. \tag{A.8}$$

Just as in the secant method or the regula falsi, this procedure is repeated until the convergence criterion is satisfied. Notice that for a successful guess of the root, the sign of the derivative of the function f must not change in the region where the points x_i are located. The convergence of this method is quadratic, that is, the scaling exponent y is equal to 2.

A4 Finding the optimum of a function

Suppose we want to locate the point where a function f assumes its minimum (for a maximum, we consider the function $-f$ and apply a minimisation procedure).

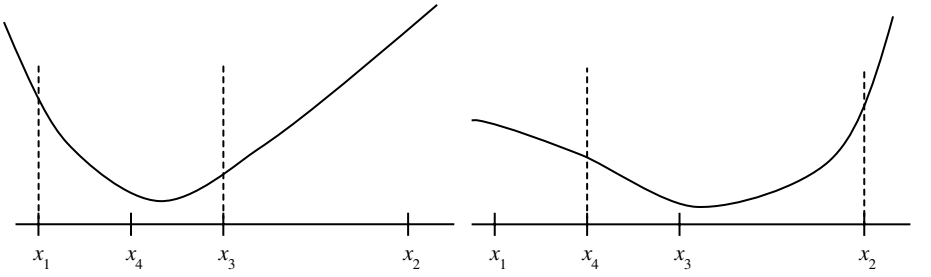


Figure A.1. Bracketing the minimum using the bisection method. The point x_4 is chosen midway between x_1 and x_3. The two graphs show two possibilities for the interval at the next step of the iteration. These intervals are indicated by the dotted lines.

We take f to depend on a single variable at first. A sufficient (but not a necessary) condition to have a local minimum in the interval $[x_1, x_2]$ is to have a point x_3 between x_1 and x_2 where f assumes a value smaller than $f(x_1)$ and $f(x_2)$:

$$x_1 < x_3 < x_2; \tag{A.9a}$$

$$f(x_3) < f(x_1) \quad \text{and} \quad f(x_3) < f(x_2). \tag{A.9b}$$

There must always exist such a triad of points closely around the local minimum. We use this condition to find an interval in which f has a minimum; we say that the interval $[x_1, x_2]$ 'brackets' the minimum. Then we can apply an algorithm to narrow this interval down to some required precision. In order to find the first bracketing interval, consider two initial points x_1 and x_2 with $f(x_2) < f(x_1)$. Then we choose a point x_3 beyond x_2 and check whether the value of f in x_3 is still smaller, or whether we are going 'uphill' again. If that is the case we have bracketed the minimum, otherwise we continue in the same manner with the points x_1 and x_3.

We consider now an algorithm for narrowing down the bracketing interval for the case that we know the derivative of the function f. Then we can use the methods of the previous section, applied to the derivative of f rather than to f itself. If the derivative is not known, we can apply a variant of the bisection method described in the previous section. The procedure is represented in Figure A.1. Suppose we have points x_1, x_2, x_3 satisfying (A.9). We take a new point x_4 either halfway in between x_1 and x_3, or between x_3 and x_2. Suppose we take the first option. Then either x_4 is the lowest point of (x_1, x_4, x_3) or x_3 is the lowest point of (x_4, x_3, x_2). In the first case, the new interval becomes $[x_1, x_3]$, with x_4 as the point in between, in the second it becomes $[x_4, x_2]$, with x_3 as the point in between, where f assumes a value lower than at the boundary points of the interval. There exist also methods for minimising a function if we do not know the derivative. Usually, more elaborate but more efficient procedures are used, which go by the name of *Brent's method*; for a description see Refs. [1, 7].

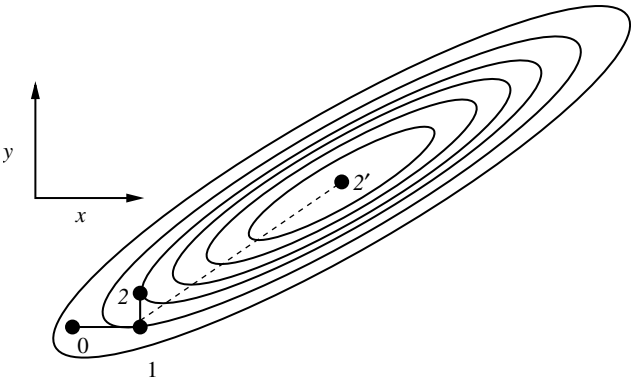


Figure A.2. Finding the minimum of a quadratic function f depending on two variables. The ellipses shown are the contour lines of the function f. The solid lines represent the steps followed in the steepest decsent method. In the conjugate gradient method, the minimum is found in two steps, the second step being the dotted line.

Next we turn to the minimisation of a function depending on more than one variable. We assume that the gradient of the function is known. To get a feeling for this problem and the solution methods, imagine that you try to reach the nearest lowest point in a mountainous landscape. We start off at some point $\mathbf{x}_0$ and we restrict ourselves to piecewise linear paths. An obvious approach is the *method of steepest descent*. Starting at some point $\mathbf{x}_0$, we walk on a straight line through x_0 along the negative gradient $\mathbf{g}_0$ at $\mathbf{x}_0$:

$$\mathbf{g}_0 = -\nabla f(\mathbf{x}_0). \tag{A.10}$$

The points $\mathbf{x}$ on this line can be parametrised as

$$\mathbf{x} = \mathbf{x}_0 + \lambda \mathbf{g}_0, \quad \lambda > 0. \tag{A.11}$$

We minimise f as a function of λ using a one-dimensional minimisation method as described at the beginning of this section. At the minimum, which we shall call $\mathbf{x}_1$, we have

$$\frac{\mathrm{d}}{\mathrm{d}\lambda} f(\mathbf{x}_0 + \lambda \mathbf{g}_0) = \mathbf{g}_0 \cdot \nabla f(\mathbf{x}_1) = 0. \tag{A.12}$$

From this point, we proceed along the negative gradient $\mathbf{g}_1$ at the point $\mathbf{x}_1$: $\mathbf{g}_1 = -\nabla f(\mathbf{x}_1)$. Note that the condition (A.12) implies that $\mathbf{g}_0$ and $\mathbf{g}_1$ are perpendicular: at each step we change direction by an angle of 90^{o}.

Although the steepest descent method seems rather efficient, it is not the best possible minimisation method. To see what can go wrong, consider the situation shown in Figure A.2, for a function depending on two variables, x and y. We start off from the point $\mathbf{x}_0$ along the negative gradient, which in Figure A.2 lies along

the x-axis. At the point $\mathbf{x}_1$, the gradient is along the y-axis:

$$\frac{\partial f}{\partial x} = 0. \tag{A.13}$$

It now remains to set the derivative along y to zero as well. To this end, we move up the y-axis, but unfortunately the gradient in the x-direction will start deviating from 0 again, so that, when we arrive at the point $\mathbf{x}_2$ where the y-gradient vanishes, we must move again in the x-direction, whereby the y-gradient starts deviating from zero and so on. Obviously, it may take a large number of steps to reduce both derivatives to zero – only in the case where the contour lines are circular will the minimum be found in two steps. The more eccentric the elliptic contours are, the more steps are needed in order to find the minimum, and the minimum is approached along a zigzag path.

To see how we can do a better job, we first realise that close to a minimum a Taylor expansion of f has the form

$$f(\mathbf{x}) = f_0 + \tfrac{1}{2}\mathbf{x} \cdot \mathbf{H}\mathbf{x} + \cdots, \tag{A.14}$$

where we have assumed that the minimum is located at the origin and that f assumes the value f_0 there (for the general case, a similar analysis applies). $\mathbf{H}$ is the *Hessian matrix*, defined as

$$H_{ij} = \frac{\partial^2 f(\mathbf{x}_0)}{\partial x_i \partial x_j}. \tag{A.15}$$

Note that $\mathbf{H}$ is symmetric if the partial derivatives of f are continuous; moreover $\mathbf{H}$ is a positive definite matrix (positive definite means that all the diagonal elements of $\mathbf{H}$ are positive), otherwise we would not be dealing with a minimum but with a saddle point or a maximum. For the moment, we restrict ourselves to quadratic functions of the form (A.14) – below we shall see how the same method can be applied to general functions. The method we describe is called the *conjugate gradient method*.

If we move from a point $\mathbf{x}_i$ along a direction $\mathbf{h}_i$, we find the point $\mathbf{x}_{i+1}$ by a line minimisation. For this point $\mathbf{x}_{i+1}$ we have

$$\mathbf{h}_i \cdot \nabla f(\mathbf{x}_{i+1}) = 0. \tag{A.16}$$

We want to move in a new direction $\mathbf{h}_{i+1}$ along which the gradient along $\mathbf{h}_i$ remains zero; therefore the gradient is allowed to change only in a direction perpendicular to $\mathbf{h}_i$. If we move from the point $\mathbf{x}_i$ over a vector $\lambda\mathbf{h}_{i+1}$, the change in the negative gradient is given by

$$\Delta\mathbf{g} = -\lambda\mathbf{H}\mathbf{h}_{i+1} \tag{A.17}$$

and we want this change to be orthogonal to the direction $\mathbf{h}_i$ of the previous line minimisation, and this implies that

$$\mathbf{h}_i \cdot \mathbf{H}\mathbf{h}_{i+1} = 0. \tag{A.18}$$

Vectors $\mathbf{h}_i$ and $\mathbf{h}_{i+1}$ satisfying this requirement are called *conjugate*. Since we have guaranteed the gradient along $\mathbf{h}_i$ to remain zero along the new path, and as the second minimisation will annihilate the gradient along $\mathbf{h}_{i+1}$ (which is independent of $\mathbf{h}_i$), at the next point the gradient is zero along two independent directions, and in the two-dimensional case we have arrived at the minimum of f: in two dimensions, only two steps are needed to arrive at the minimum of a quadratic function. For n dimensions, we need n steps to arrive at the minimum. This is less obvious than it seems. Remember that the second step (along $\mathbf{h}_{i+1}$) was designed such that it does not spoil the gradient along the direction $\mathbf{h}_i$. The third step will be designed such that it does not spoil the gradient along the direction $\mathbf{h}_{i+1}$. But does it leave the gradient along $\mathbf{h}_i$ unchanged as well? For this to be true at all stages of the iteration, *all* the directions $\mathbf{h}_i$ should be conjugate:

$$\mathbf{h}_i \cdot \mathbf{H}\mathbf{h}_j = 0, \text{all} i \neq j. \tag{A.19}$$

If $\mathbf{H}$ is the unit matrix, this requirement is equivalent to the set $\{\mathbf{h}_i\}$ being orthogonal. For a symmetric and positive Hessian matrix, the set $\{\mathbf{H}^{1/2}\mathbf{h}_i\}$ is orthogonal for a conjugate set $\{\mathbf{h}_i\}$.

To see perhaps more clearly where the magic of the conjugate gradients arises from, we define new vectors $\mathbf{y}_i = \sqrt{\mathbf{H}}\mathbf{h}_i$. We can always take the square root of $\mathbf{H}$ as this matrix has positive, real eigenvalues (see also Section 3.3). In terms of these new vectors we have the trivial problem of minimising:

$$\mathbf{y} \cdot \mathbf{y}. \tag{A.20}$$

Starting at some arbitrary point, we can simply follow a sequence of orthogonal directions along each of which we minimise this expression. Now it is trivial that these minimisations do not spoil each other. But the fact that the vectors in the set y_i are othogonal implies

$$\mathbf{h}_i \cdot \mathbf{H}\mathbf{h}_j = 0 \tag{A.21}$$

for $i \neq j$, that is, the directions $\mathbf{h}_i$ form a conjugate set. Each sequence of conjugate search directions therefore corresponds to a sequence of orthogonal search directions $\mathbf{y}_i$.

How can we construct a sequence of conjugate directions? It turns out that the following recipe works. We start with the negative gradient $\mathbf{g}_0$ at $\mathbf{x}_0$ and we take $\mathbf{h}_0 = \mathbf{g}_0$. The following algorithm constructs a new set of vectors $\mathbf{x}_{i+1}$, $\mathbf{g}_{i+1}$, $\mathbf{h}_{i+1}$ from the previous one ($\mathbf{x}_i$, $\mathbf{g}_i$, $\mathbf{h}_i$):

$$\mathbf{g}_{i+1} = \mathbf{g}_i - \lambda_i \mathbf{H}\mathbf{h}_i; \tag{A.22a}$$

$$\mathbf{h}_{i+1} = \mathbf{g}_{i+1} + \gamma_i \mathbf{h}_i; \tag{A.22b}$$

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \lambda_i \mathbf{h}_i \tag{A.22c}$$

with

$$\lambda_i = \frac{\mathbf{g}_i \cdot \mathbf{g}_i}{\mathbf{g}_i \cdot \mathbf{H}\mathbf{h}_i}; \tag{A.23a}$$

$$\gamma_i = -\frac{\mathbf{g}_{i+1} \cdot \mathbf{H}\mathbf{h}_i}{\mathbf{h}_i \cdot \mathbf{H}\mathbf{h}_i}. \tag{A.23b}$$

For this algorithm, the following properties can be verified:

(i) $\mathbf{g}_i \cdot \mathbf{g}_{i+1} = 0$ for all i .
(ii) $\mathbf{h}_{i+1} \cdot \mathbf{H}\mathbf{h}_i = 0$, for all i.
(iii) $\mathbf{g}_{i+1} \cdot \mathbf{h}_i = 0$ for all i (this is equivalent to (A.16)).
(iv) $\mathbf{g}_i \cdot \mathbf{g}_j = 0$ for all $i \neq j$.
(v) $\mathbf{h}_i \cdot \mathbf{H}\mathbf{h}_j = 0$ for all $i \neq j$.

The proof of these statements will be considered in Problem A.2. It is now easy to derive the following alternative formulas for λ_i and γ_i:

$$\lambda_i = \frac{\mathbf{g}_i \cdot \mathbf{h}_i}{\mathbf{h}_i \cdot \mathbf{H}\mathbf{h}_i}; \tag{A.24a}$$

$$\gamma_i = \frac{\mathbf{g}_{i+1} \cdot \mathbf{g}_{i+1}}{\mathbf{g}_i \cdot \mathbf{g}_i}. \tag{A.24b}$$

For an arbitrary (i.e. nonquadratic) function, the conjugate gradient method will probably be less efficient. However, it is expected to perform significantly better than the steepest descent method, in particular close to the minimum where f can be approximated well by a quadratic form. A problem is that the above prescription cannot be used directly as we do not know the matrix $\mathbf{H}$. However, in this case λ_i can be found from a line minimisation: x_{i+1} should be such that it is a minimum of f along the line $\mathbf{x}_i + \lambda_i \mathbf{h}_i$, in other words $\mathbf{h}_i \cdot \nabla f(\mathbf{x}_i + \lambda_i \mathbf{h}_i) = 0$. For a quadratic function as considered above, this condition does indeed reduce to (A.24a); for a general function we use the bisection minimisation or Brent's method. We know then the value of $\mathbf{x}_{i+1}$. The gradients $\mathbf{g}_{i+1}$ can be calculated directly as $\mathbf{g}_{i+1} = -\nabla f(\mathbf{x}_{i+1})$. We use $\mathbf{g}_{i+1}$ and $\mathbf{g}_i$, together with equation (A.24b) to find γ_{i+1}, and use this in (A.22b) to find $\mathbf{h}_{i+1}$. We see that the function values and gradients of f suffice to construct a sequence of directions which in the case of a quadratic function are conjugate.

A5 Discretisation

In the equations of physics, most quantities are functions of continuous variables. Obviously, it is impossible to represent such functions numerically in a computer since real numbers are always stored using a finite number of bits (typically 32 or 64) and therefore only a finite number of different real numbers is available. But even storing a function for all arguments allowed by the computer resolution is

impossible since the memory required for this would exceed any reasonable bounds. Instead of representing the functions with the maximum precision possible, they are usually defined on a much coarser grid, specified by the user and independent of the representation of real numbers in the computer. Most problems can be solved with sufficient accuracy using discretised variables. One should, however, in all cases be careful in choosing the discretisation step; a balance must be found between the number of values and the level of precision .

Suppose we want to solve Newton's equations for the motion of a satellite orbiting around the Earth. Its path is quite smooth, the velocity changes relatively slowly so that a relatively large and constant time interval yields accurate results. If we consider a rocket launched from the Earth which should orbit around the Moon, most of the path between Earth and Moon will be smooth so that a large time step is possible, but when the rocket comes close to the Moon, its velocity may change strongly because of the Moon's attraction, and a smaller time step will then be necessary to keep the representation accurate.

Very many numerical methods are based on discretisation. If a function is represented on a discrete grid, it is possible to reconstruct the function by interpolation. Interpolation often consists of constructing a polynomial that assumes the same value as the discretised function on the grid points. The larger the number of points taken into account, the higher the order of the interpolation polynomial. A higher order implies a more accurate interpolation, but too high an order often results in strongly oscillatory behaviour of the polynomial between the grid points, so that it deviates strongly from the original function there. Interpolations are often used to derive numerical methods with a high order of accuracy. Examples can be found in the next two sections. When discretising a function, it should always be kept in mind that the interval must be chosen such that the main features of the function are preserved in the discretised representation.

A6 Numerical quadratures

Numerical integration (or *quadrature*) of a continuous and bounded function on the interval $[a, b]$ can be done straightforwardly. One defines an equidistant grid on the interval $[a, b]$: the grid points x_n are given by

$$x_n = a + nh \tag{A.25}$$

with $h = (b - a)/N$ and the index n running from 0 to N. It is clear that a (crude) approximation to the integral is given by the sum of the function values on the grid points:

$$\int_a^b f(x)\mathrm{d}x \approx h \sum_{n=0}^{N-1} f(x_n). \tag{A.26}$$

Such an approximation is useless without an estimate of the error in the result. If f is continuous and bounded on the interval $[x_n, x_n + h]$, its value does not deviate from $f(x_n)$ more than Mh, with M some finite constant, depending on the function f and the values of a and b. Therefore, the relative error in this result is $\mathcal{O}(h)$: doubling the number of integration points reduces the error by a factor of about 2.

A way of approximating the integral of a differentiable function f to second order in h using the grid points given in Eq. (A.25) is the following. Approximate f on the interval $[x_n, x_{n+1}]$ to first order in h by

$$f(x_n) + \frac{(x - x_n)}{h}[f(x_{n+1}) - f(x_n)] + \mathcal{O}(h^2), \; n = 0, \ldots, N-1. \tag{A.27}$$

Integrating this form analytically on the interval, we obtain

$$\int_a^b f(x)\mathrm{d}x = h\left[\frac{1}{2}f(x_0) + f(x_1) + f(x_2) + \cdots + f(x_{N-1}) + \frac{1}{2}f(x_N)\right] + \mathcal{O}(h^2). \tag{A.28}$$

This is called the trapezoidal rule.

If f is twice differentiable, we can approximate it by piecewise quadratic functions. For a single interval using three equidistant points, we have

$$\int_a^b f(x)\mathrm{d}x = \frac{b-a}{6}\left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)\right] + \mathcal{O}[(b-a)^5], \tag{A.29}$$

which is one order of magnitude better than expected, because there is a cancellation of errors at the left and right hand boundaries, resulting from the symmetry of this formula. If the interval $[a, b]$ is split up into $N/2$ subintervals, each of size $2h$, we obtain the following expression:

$$\int_a^b f(x)\mathrm{d}x = h\frac{1}{3}[f(x_0) + 4f(x_1) + 2f(x_3) + 4f(x_4) + \cdots + 2f(x_{N-2}) + 4f(x_{N-1}) + f(x_N)] + \mathcal{O}(h^4). \tag{A.30}$$

This is an example of *Simpson's rule*.

It would be easy to extend this list of algorithms. They all boil down to making some polynomial approximation of the integrand and then integrating the approximate formula analytically, which is trivial for a piecewise polynomial. Also, many tricks exist for integrating functions containing singularities, but these are beyond the scope of this discussion. In all cases, the order of the accuracy is equal to the number of points used in fitting the polynomial, since for n points one can determine a polynomial of order $n - 1$ having the same value as the function to be integrated on all n points.

A very efficient method consists of repeating the trapezoidal rule and performing it for successive values of h, each having half the size of the previous one. This yields a sequence of approximants to the integral for various values of h. These

approximants can be fitted to a polynomial and the value for this polynomial at $h = 0$ is a very accurate approximation to the exact value. This is called the *Romberg method*.

The famous Gauss integration method works essentially the same way as the polynomial methods for equidistant points described above, but for the grid points x_n the zeroes of the Legendre polynomials are taken, and on the interval $[a, b]$ the function f is approximated by Legendre polynomials. Legendre polynomials P_l have the property of being orthonormal on the interval $[-1, 1]$:

$$\int_{-1}^{1} P_l(x)P_{l'}(x)\mathrm{d}x = \delta_{ll'}. \tag{A.31}$$

The advantage of the Gauss–Legendre method is that its accuracy is much better than that of other methods using the same number of integration points: the accuracy of an N-point Gauss–Legendre method is equivalent to that of an equidistant-point method using $2N$ points!

We give no derivations but just present the resulting algorithm on the interval $[-1, 1]$:

$$\int_{-1}^{1} \mathrm{d}x\, f(x) = \sum_{n=1}^{N} w_n f(x_n) + \mathcal{O}(h^{2N}). \tag{A.32}$$

Here, x_n are the zeroes of the Legendre polynomial P_N, h is $2/N$, x_n and w_n can be found in many books. Moreover, there exist programs to generate them [1].

There exist other Gauss integration methods for nonbounded intervals [1].

A7 Differential equations

Many physical theories boil down to one or more differential equations: for example Newton's second law, the Schrödinger equation or Maxwell's equations. It is therefore of primary importance to the computational physicist to have available reliable and efficient methods for solving such equations. As we have seen in Chapter 3 of this book, we can often determine the stationary functions of a functional instead of solving the differential equations directly (these equations can even be derived using such a stationarity condition), but here we restrict ourselves to the direct solution of the differential equations. In the field of numerical analysis, many methods have been developed to solve differential equations. Here we shall not treat these methods in great detail, but review the most common ones and their properties.

What makes a method 'good' depends on the problem at hand, and a number of criteria can be distinguished:

- **Precision and speed**. Higher precision will generally cost more time, higher speed will yield less precise results.

- **Stability**. In some methods, errors in the starting values or errors due to the discrete numerical representation tend to grow during integration, so that the solutions obtained deviate sometimes strongly from the exact ones. If the errors tend to grow during integration, the method is called *unstable*. It is essentially the same phenomenon as we encountered in the discussion of recursion in Appendix A2.
- **Implementation**. Very complicated algorithms are sometimes less favourable because of the time needed to implement them correctly. This criterion is of course irrelevant when using existing routines or programs (e.g. from numerical libraries).
- **Flexibility**. In all methods, the coordinates are discretised: some methods demand a fixed discretisation interval. These are less useful for problems with solutions having strongly varying behaviour (somewhere very smooth and elsewhere oscillating rapidly).
- **Symmetry**. For particular types of differential equations we would like the numerical method to share symmetry properties of the original equation; an example is time reversibility which might be present in the equation of motion of a particle. In Chapter 8, symplectic symmetry properties of Hamiltonian equations and particular integration schemes are discussed.

There are other criteria, such as analyticity of the functions occurring in the differential equation, which make some methods more suitable than others.

A7.1 Ordinary differential equations

We now describe a number of numerical algorithms for the solution of this type of equation. In one dimension, a first order differential equation looks like

$$\dot{x}(t) = f[x(t), t]. \tag{A.33}$$

We call the variables x and t 'space' and 'time' respectively, although in various problems they will represent completely different quantities. In the following we integrate always from $t = 0$. In practice, one integrates from arbitrary values of t, but the methods described here are trivially generalised.

By writing $y(t) = \dot{x}(t)$, a second order equation

$$\ddot{x}(t) = f[x(t), t] \tag{A.34}$$

can be transformed into two differential equations of the form (A.33):

$$\dot{x}(t) = y(t) \tag{A.35a}$$

$$\dot{y}(t) = f[x(t), t] \tag{A.35b}$$

and the methods that will be discussed are easily generalised to this two-dimensional case.

Stability

As noted above, some differential equations are susceptible to instabilities in the numerical solution. In first order, one-dimensional equations, taking a small time step usually guarantees stability, but in higher dimensions or for higher orders it is not so easy to avoid instabilities. The reason is that such equations have in general more than one independent solution; the initial values determine how these are combined into the actual solution. The situation often occurs that the solution we are after is a damped one (in the direction in which we integrate), but there exists a growing solution to the equation too which will mix into our numerical solution and spoil the latter as it grows, just as in the recursion problems discussed in Appendix A2. If possible, one should integrate in the other direction with the appropriate initial conditions, so that the required solution is the growing one, as instabilities are absent in that case.

Sometimes, however, the problem cannot be solved so easily, and this is particularly the case when the two solutions have a very different time scale. There might, for example, be a combination of a fast and a slow oscillation, where the time scale of the fast one determines the time step for which the integration is stable. More tricky is the existence of a very strongly damped term in the solution, which is easily overlooked (because it damps out so quickly) but might spoil the numerical solution if the time step used is not adapted to the strongly damped term. Equations that suffer from this type of problem are called 'stiff equations' [4]. For more details about stiff equations see Ref. [4].

The Runge–Kutta method

This is a frequently used method for the solution of ordinary differential equations. Like most methods for solving differential equations, the Runge–Kutta method can be considered as a step-wise improvement of Euler's forward method, the latter predicting the solution to the differential equation (A.33), stating that, given $x(0)$, $x(t)$ for $t = h$ is given by

$$x(h) = x(0) + hf[x(0), 0] + \mathcal{O}(h^2). \tag{A.36}$$

It is clear that if we use Euler's rule halfway across the interval $(0, h)$ to estimate $x(h/2)$, and evaluate $f(x, t)$ for this value of x and $t = h/2$, we obtain a better estimate for $x(h)$:

$$\begin{aligned} k_1 &= hf[x(0), 0] \\ k_2 &= hf\left[x(0) + \frac{1}{2}k_1, h/2\right] \\ x(h) &= x(0) + k_2 + \mathcal{O}(h^3). \end{aligned} \tag{A.37}$$

This is called the *midpoint rule*. The fact that the error is of order h^3 can be verified by expanding $x(t)$ in a power series in t to second order. One can systematically derive similar rules for higher orders. The most popular one is the fourth order rule:

$$\begin{aligned} k_1 &= hf(x,0) \\ k_2 &= hf\left(x+\frac{1}{2}k_1, h/2\right) \\ k_3 &= hf\left(x+\frac{1}{2}k_2, h/2\right) \\ k_4 &= hf(x+k_3, h) \\ x(h) &= x(0)+\frac{1}{6}(k_1+2k_2+2k_3+k_4)+\mathcal{O}(h^5). \end{aligned} \tag{A.38}$$

We shall not present a derivation of this rule, because it involves some tedious algebra. Although the name 'Runge–Kutta method' is used for the method for arbitrary order, it is usually reserved for the fourth order version.

The Runge–Kutta method owes its popularity mostly to the possibility of using a variable discretisation step: at every step, one can decide to use a different time interval without difficulty. A severe disadvantage for some applications is the fact that the function $f(x,t)$ has to be evaluated four times per integration step. Especially when integrating Newton's equations for many-particle systems, as is done in molecular dynamics, the force evaluations demand a major part of the computer time, and the Runge–Kutta method with its large number of force evaluations becomes therefore very inefficient.[1]

It is possible to implement the Runge–Kutta method with a prescribed maximum error, δ, to the resulting solution, by adapting the step size automatically. In this method the fact that the Runge–Kutta rule has an order h^5 error is used to predict for which time step the desired accuracy will be achieved. For each time step t, the value of x at time $t+h$ is calculated twice, first using a time interval h and then using two intervals, each of length $h/2$. By comparing the two results, it is possible to estimate the time step needed to determine the resulting value with the prescribed precision δ. If the absolute value of the difference between the two results is equal to Δ, the new time step h' is given by $h' = (15/16)h(\delta/\Delta)^{1/5}$, as can easily be verified. It is recommended to use the Runge–Kutta rule only in this implementation, because when using a constant time step the user gets no warning whatsoever about this step being too large (for example when the function f is varying strongly) and the solution may deviate appreciably from the exact one without this being noticed.

[1] The fact that Runge–Kutta methods are not symplectic is another reason why they are not suitable for this application – see Chapter 8.

Verlet algorithm

The Verlet algorithm is a simple method for integrating second order differential equations of the form

$$\ddot{x}(t) = F[x(t), t]. \tag{A.39}$$

It is used very often in molecular dynamics simulations because of its simplicity and stability (see Chapter 8). It had been mentioned already at the end of the eighteenth century [8, 9]. It has a fixed time discretisation interval and it needs only one evaluation of the 'force' F per step. The Verlet algorithm is easily derived by adding the Taylor expansions for the coordinate x at $t = \pm h$ about $t = 0$:

$$x(h) = x(0) + h\dot{x}(0) + \frac{h^2}{2}F[x(0), 0] + \frac{h^3}{6}\dddot{x}(0) + \mathcal{O}(h^4) \tag{A.40}$$

$$x(-h) = x(0) - h\dot{x}(0) + \frac{h^2}{2}F[x(0), 0] - \frac{h^3}{6}\dddot{x}(0) + \mathcal{O}(h^4) \tag{A.41}$$

leading to

$$x(h) = 2x(0) - x(-h) + h^2F[x(0), 0] + \mathcal{O}(h^4). \tag{A.42}$$

Knowing the values of x at time 0 and $-h$, this algorithm predicts the value of $x(h)$. We always need the last two values of x to produce the next one. If we only have the initial position, $x(0)$, and velocity, $v(0)$, at our disposal, we approximate $x(h)$ by

$$x(h) = x(0) + v(0)h + \frac{h^2}{2}F[x(0), 0] + \mathcal{O}(h^3). \tag{A.43}$$

The algorithm is invariant under time reversal (as is the original differential equation). This means that if, after having integrated the equations for some time, time is reversed by swapping the two most recent values of x, exactly the same path in phase space should be followed backward in time. In practice, there will always be small differences as a result of finite numerical precision of the computer.

As we shall see in Problem A.3, the accumulated error in the position after a large number of integration steps is of order $\mathcal{O}(h^2)$. As there is no point in calculating the velocities with higher precision than that of the positions, we can evaluate the velocities using the simple formula

$$v(0) = \frac{x(h) - x(-h)}{2h} \tag{A.44}$$

which gives the velocity with the required $\mathcal{O}(h^2)$ error.

Using the velocities at half-integer time steps:

$$v[(n + 1/2)h] = \{x[(n + 1)h] - x(nh)\}/h + \mathcal{O}(h^2), \tag{A.45}$$

we may reformulate the Verlet algorithm in the *leap-frog* form:

$$v(h/2) = v(-h/2) + hF[x(0), 0] \tag{A.46a}$$

$$x(h) = x(0) + hv(h/2). \tag{A.46b}$$

Note that this form is exactly equivalent to the Verlet form, so the error in the positions is still $\mathcal{O}(h^4)$. This form is less sensitive to errors resulting from finite precision arithmetic in the computer than the Verlet form (A.42).

Note that the time step is not easily adapted in the Verlet/leap-frog algorithm. This is, however, possible, either by changing the coefficients of the various terms in the equation, or by reconstructing the starting values with the new time step by interpolation.

Consider the Verlet solution for the harmonic oscillator $\ddot{x} = -Cx$:

$$x(t + h) = 2x(t) - x(t - h) - h^2 Cx(t). \tag{A.47}$$

The analytic solution to this algorithm (not the exact solution to the continuum differential equation) can be written in the form $x(t) = \exp(i\omega t)$, with ω satisfying

$$2 - 2\cos(\omega h) = h^2 C. \tag{A.48}$$

If $h^2 C > 4$, ω becomes imaginary, and the analytical solution becomes unstable. Of course we would not take h that large in actual applications, as we always take it substantially smaller than the period of the continuum solution. It is, however, useful to be aware of this instability, especially when integrating systems of second order differential equations, which reduce to a set of coupled harmonic equations close to the stationary solutions. High-frequency degrees of variables are then often easily overlooked.

Numerov's method

This is an efficient method for solving equations of the type

$$\ddot{x}(t) = f(t)x(t) \tag{A.49}$$

of which the stationary Schrödinger equation is an example [10]. Numerov's method makes use of the special structure of this equation in order to have the fourth order contribution to $x(h)$ cancel, leading to a form similar to the Verlet algorithm, but accurate to order h^6 (only even orders of h occur because of time-reversal symmetry). The Verlet algorithm (see above) was derived by expanding $x(t)$ up to second order around $t = 0$ and adding the resulting expression for $t = h$ and for $t = -h$. If we do the same for Eq. (A.49) but now expand $x(t)$ to order six in t, we

obtain

$$x(h) + x(-h) - 2x(0) = h^2 f(0)x(0) + \frac{h^4}{12}x^{(4)}(0) + \frac{h^6}{360}x^{(6)}(0) + \mathcal{O}(h^8) \tag{A.50}$$

with $x^{(4)}$ being the fourth and $x^{(6)}$ the sixth derivative of x with respect to t. As these derivatives are not known, this formula is not useful as such. However, we can evaluate the fourth derivative as the discrete second derivative of the second derivative of $x(t)$ taken at $t = 0$. Using the differential equation (A.49) we then obtain

$$x^{(4)}(0) = \frac{x(h)f(h) + x(-h)f(-h) - 2x(0)f(0)}{h^2}, \tag{A.51}$$

and then, after switching to another variable $w(t) = [1 - (h^2/12)f(t)]x(t)$, Eq. (A.50) becomes

$$w(h) + w(-h) - 2w(0) = h^2 f(0)x(0) + \mathcal{O}(h^6). \tag{A.52}$$

Now we see that, using a second order integration scheme for w (i.e. using only two values of the solution to predict the next one), $x(h)$ is known to order h^6. Whenever $x(t)$ is required, it can be calculated as $x(t) = [1 - (h^2/12)f(t)]^{-1}w(t)$. Note that the integration error over a fixed interval scales as h^4, that is, two orders less than the single-step error – see the analysis in Problem A.3.

The implementation of the initial conditions is not straightforward: when not dealt with carefully enough, we might obtain errors which at the end of the integration interval scale worse than $\mathcal{O}(h^4)$. Usually the initial value $x(0)$ and the derivative $\dot{x}(0)$ are known. From these, we need to predict the value at $x(h)$ with an accuracy of at least $\mathcal{O}(h^6)$. This can be done by first subtracting the Taylor expansions for $x(h)$ and $x(-h)$ rather than adding them as was done in the derivation of the Numerov algorithm. The result is

$$2h\dot{x}(0) = [1 - h^2 f(h)/6]x(h) - [1 - h^2 f(-h)/6]x(-h) + \mathcal{O}(h^5). \tag{A.53}$$

The derivative $\dot{x}(0)$ is known: together with the Numerov algorithm, we have two equations for $x(h)$ and $x(-h)$. Eliminating the latter, we are left with

$$x(h) = \frac{[2 + 5h^2 f(0)/6][1 - h^2 f(-h)/12]x(0) + 2h\dot{x}(0)[1 - h^2 f(-h)/6]}{[1 - h^2 f(h)/12][1 - h^2 f(-h)/6] + [1 - h^2 f(-h)/12][1 - h^2 f(h)/6]}. \tag{A.54}$$

Starting with $x(0)$ and $x(h)$, the Numerov algorithm can be applied straightforwardly.

More details about this method and its applications can be found in Ref. [11].

Finite difference methods

The Verlet algorithm is a special example of this class of methods. Finite difference algorithms exist for various numerical problems, like interpolation, numerical differentiation and solving ordinary differential equations [12]. Here we consider the solution of a differential equation of the form

$$\dot{x}(t) = f[x(t), t], \tag{A.55}$$

and in passing we touch upon the interpolation method.

For any function x depending on the coordinate t, one can build the following table (the entries in this table will be explained below):

-3	x_{-3}						
		$\delta x_{-5/2}$					
-2	x_{-2}		$\delta^2 x_{-2}$				
		$\delta x_{-3/2}$		$\delta^3 x_{-3/2}$			
-1	x_{-1}		$\delta^2 x_{-1}$		$\delta^4 x_{-1}$		
		$\delta x_{-1/2}$		$\delta^3 x_{-1/2}$		$\delta^5 x_{-1/2}$	
0	x_0		$\delta^2 x_0$		$\delta^4 x_0$		$\delta^6 x_0$
		$\delta x_{1/2}$		$\delta^3 x_{1/2}$		$\delta^5 x_{1/2}$	
1	x_1		$\delta^2 x_1$		$\delta^4 x_1$		
		$\delta x_{3/2}$		$\delta^3 x_{3/2}$			
2	x_2		$\delta^2 x_2$				
		$\delta x_{5/2}$					
3	x_3						

The first column contains equidistant time steps at separation h, and the second one the values x_t for the times t of the first column. We measure the time in units of h. The third column contains the differences between two subsequent values of the second one; therefore they are written just halfway between these two values. The fourth column contains differences of these differences and so on. Formally, except for the first column, the entries are defined by

$$\begin{aligned} \delta^j x_{i-1/2} &= \delta^{j-1} x_i - \delta^{j-1} x_{i-1} \\ \delta^j x_i &= \delta^{j-1} x_{i+1/2} - \delta^{j-1} x_{i-1/2} \quad \text{and} \\ \delta^0 x_i &\equiv x_i. \end{aligned} \tag{A.56}$$

It is possible to build the entire table from the second column only, or from the values $\delta^{2j} x_0$ and $\delta^{2j+1} x_{1/2}$ (that is, the elements on the middle horizontal line and the line just below), because the table is highly redundant.

Such a table can be used to construct an interpolation polynomial for x_t. First we note that

$$\begin{aligned} x_{\pm 1} &= x_0 \pm \delta x_{\pm 1/2} \\ x_{\pm 2} &= x_0 \pm 2\delta x_{\pm 1/2} + \delta^2 x_{\pm 1} \\ x_{\pm 3} &= x_0 \pm 3\delta x_{\pm 1/2} + 3\delta^2 x_{\pm 1} \pm \delta^3 x_{\pm 3/2} \end{aligned} \tag{A.57}$$

and so on. In these expressions the binomial coefficients are recognised. From these equations, it can be directly seen that

$$x_t = x_0 + t\delta x_{1/2} + \frac{t(t-1)}{2}\delta^2 x_1 + \frac{t(t-1)(t-2)}{3!}\delta^3 x_{3/2} + \cdots \tag{A.58}$$

is a polynomial which coincides with x_t for $t = 1, 2, 3$. One can also build higher order polynomials in the same fashion. For negative t-values, this formula reads

$$x_t = x_0 + t\delta x_{-1/2} + \frac{t(t+1)}{2}\delta^2 x_{-1} + \frac{t(t+1)(t+2)}{3!}\delta^3 x_{-3/2} + \cdots \tag{A.59}$$

These formulas are called *Newton interpolation formulas*.

We now use this table to integrate differential equations of type (A.55). Therefore we consider a difference table, not for x, but for $\dot{x}$:

$$\begin{array}{lllll} -2 & \dot{x}_{-2} & & \delta^2\dot{x}_{-2} & \\ & & \delta\dot{x}_{-3/2} & & \delta^3\dot{x}_{-3/2} \\ -1 & \dot{x}_{-1} & & \delta^2\dot{x}_{-1} & \\ & & \delta\dot{x}_{-1/2} & & \\ 0 & \dot{x}_0 & & & \\ & & & & \\ 1 & & & & \end{array}$$

For simplicity we did not make this table too 'deep' (up to third differences). Suppose the equation has been integrated up to $t = 1$, so x_1 is known, and we would like to calculate x_2. This is possible by adding an extra row to the lower end of the table. This can be done because from (A.55), $\dot{x}_1 = f(x_1, 1)$ and the table can be extended as follows:

$$\begin{array}{lllll} -2 & \dot{x}_{-2} & & \delta^2\dot{x}_{-2} & \\ & & \delta\dot{x}_{-3/2} & & \delta^3\dot{x}_{-3/2} \\ -1 & \dot{x}_{-1} & & \delta^2\dot{x}_{-1} & \\ & & \delta\dot{x}_{-1/2} & & \delta^3\dot{x}_{-1/2} \\ 0 & \dot{x}_0 & & \delta^2\dot{x}_0 & \\ & & \delta\dot{x}_{1/2} & & \\ 1 & \dot{x}_1 & & & \end{array}$$

Using this table, an interpolation polynomial for $\dot{x}_t$ can be constructed. The Newton interpolation polynomial (A.59) for $\dot{x}$ reads

$$\dot{x}_t = \dot{x}_1 + (t-1)\delta\dot{x}_{1/2} + \frac{(t-1)t}{2!}\delta^2\dot{x}_0 + \frac{(t-1)(t+1)t}{3!}\delta^3\dot{x}_{-1/2} + \cdots \quad \text{(A.60)}$$

We can now use this as an extrapolation polynomial to calculate x_2 (we assumed x_1 to be known). Using

$$x_2 = x_1 + \int_1^2 \dot{x}_t \mathrm{d}t, \quad \text{(A.61)}$$

one finds, using (A.60):

$$x_2 = x_1 + \dot{x}_1 + \frac{1}{2}\delta\dot{x}_{1/2} + \frac{5}{12}\delta^2\dot{x}_0 + \frac{3}{8}\delta^3\dot{x}_{-1/2}. \quad \text{(A.62)}$$

This value can then be used to add another row to the table, and so on.

Up to order 6, the formula reads

$$x_2 = x_1 + \dot{x}_1 + \frac{1}{2}\delta\dot{x}_{1/2} + \frac{5}{12}\delta^2\dot{x}_0 + \frac{3}{8}\delta^3\dot{x}_{-1/2} + \frac{251}{720}\delta^4\dot{x}_{-1} + \frac{95}{288}\delta^5\dot{x}_{-3/2} + \frac{19087}{60480}\delta^6\dot{x}_{-2} + \cdots \quad \text{(A.63)}$$

This equation is known as Adams' formula. Note that during the integration it is needed to renew the lower diagonal of the table only and therefore we need not store the entire table in memory.

To start the algorithm, we need the solution to the differential equation at the first few time steps to be able to set up the table. This solution can be generated using a Runge–Kutta starter, for example. The starting points should, however, be of the same order of accuracy as Adams' method itself. If this is not the case, we can use a special procedure which improves the accuracy of the table iteratively [13].

Finite difference methods are often very efficient as they can be of quite high order. As soon as the integration becomes inaccurate because of too large a time step, the deepest (i.e. the rightmost) differences tend to diverge and will lead to an overflow. This means that absence of such trouble implies high accuracy.

One can also construct so-called *predictor-corrector* methods in this way. In these methods, after every integration step the last value of x_t (in our case this is x_2) is calculated again, now using also $\dot{x}_t$ which is determined by the value of x_t and the differential equation (A.55). This procedure is repeated until the new value for x_t is close enough to the previous one. Predictor-corrector methods are seldom used nowadays, since using a sufficiently small integration step makes the time-consuming corrector cycle superfluous.

Bulirsch–Stoer method

This method is similar to the Romberg method for numerical integration. Suppose the value of x_t is wanted, x_0 being known. Over the interval $[0, t]$, the equation can be integrated using a simple method with a few steps. Then the method is repeated with a larger number of steps and so on. The resulting predictions for x_t are stored as a function of the inverse of the number of steps used. Then these values are used to build an interpolation polynomial which is extrapolated to an infinite number of steps. The efficiency of this method is another reason why predictor-corrector methods are seldom used.

A7.2 Partial differential equations

In mathematics, one usually classifies partial differential equations (PDE) according to their characteristics, leading to three different types: parabolic, elliptic and hyperbolic. In numerical analysis, this classification is less relevant, and only two different types of equations are distinguished: initial value and boundary value problems. We study examples of both categories. In the next two sections, we describe finite difference methods for these two types and then we discuss several other methods for partial differential equations very briefly.

Initial value problems

An important example of this class of problems is the diffusion equation or the time-dependent Schrödinger equation (see Section 12.2.4 for a discussion of the relation between these two equations) which is mathematically the same as the diffusion equation, the only difference being a factor i before the time derivative. In the following, we shall consider the Schrödinger equation, but the analysis is the same for the diffusion equation.

Consider the one-dimensional time-dependent Schrödinger equation (we take $\hbar = 2m = 1$):

$$\mathrm{i}\frac{\partial \psi(x,t)}{\partial t} = -\frac{\partial^2 \psi(x,t)}{\partial x^2} + V(x)\psi(x,t), \tag{A.64}$$

or, in a more compact notation:

$$\mathrm{i}\frac{\partial \psi(x,t)}{\partial t} = H\psi(x,t) \tag{A.65}$$

with H standing for the Hamilton operator. First we discretise H at L equidistant positions on the real axis, with separation Δx. The value of ψ at the point $x_j = j\Delta x$, $j = 1, \ldots, L$, is denoted by ψ_j. We can then approximate the second order derivative

with respect to x on the jth position as follows:

$$\frac{\partial^2\psi(x_j,t)}{\partial x^2} = \frac{1}{\Delta x^2}[\psi_{j-1}(t) + \psi_{j+1}(t) - 2\psi_j(t)] + \mathcal{O}(\Delta x^2). \tag{A.66}$$

We denote the discretised Hamiltonian by H_{D}, so that the Schrödinger equation can be written as

$$\mathrm{i}\frac{d\psi_j(t)}{dt} = H_{\mathrm{D}}\psi_j(t). \tag{A.67}$$

As a next step, we discretise time using intervals Δt. Indicating the nth time step with an upper index n to the wave function ψ, the time derivative of ψ_j^n at this time step can be approximated by $(\psi_j^{n+1} - \psi_j^n)/\Delta t$ (with an error of order Δt), so that the discretised form of Eq. (A.64) is given by

$$\mathrm{i}\frac{1}{\Delta t}(\psi_j^{n+1} - \psi_j^n) = \frac{1}{\Delta x^2}[-\psi_{j-1}^n(t) - \psi_{j+1}^n(t) + 2\psi_j^n(t)] + V_j\psi_j^n \tag{A.68}$$

or, in shorthand notation:

$$\psi_j^{n+1} = (1 - \mathrm{i}\Delta t H_{\mathrm{D}})\psi_j^n. \tag{A.69}$$

The stability of this method can be investigated using the so-called Von Neumann analysis. In this analysis, one considers the analytical solution to Eq. (A.69) for V_j constant:

$$\psi_j^n = \xi^n \exp(\mathrm{i}kj\Delta x). \tag{A.70}$$

The wave vector k depends on the boundary conditions, and ξ and k are related – the relation can be found by substituting this solution in Eq. (A.68). If there exists such a solution with $|\xi| > 1$, it follows that a small perturbation in the solution tends to grow exponentially, because an expansion of a generic small perturbation in terms of the solutions (A.70) will almost certainly contain the $|\xi| > 1$ modes. For our algorithm, we find

$$\xi = 1 - \mathrm{i}\Delta t\left[\frac{4}{\Delta^2 x}\sin^2(k\Delta x/2) + V_j\right] \tag{A.71}$$

which means that in all cases $|\xi| > 1$, indicating that this method is always unstable. Therefore, we shall consider a modification of the method.

First, the wave function occurring in the right hand side of (A.69) is calculated at time $t = (n+1)\Delta t$. We then arrive at a form which reads in shorthand:

$$\psi_j^{n+1} = \psi_j^n - \mathrm{i}\Delta t H_{\mathrm{D}}\psi_j^{n+1}. \tag{A.72}$$

This algorithm seems impractical, however, because to determine ψ^{n+1} on the left hand side of (A.72) it is needed on the right hand side: it is an *implicit* form. We can

make this explicit by an inversion:

$$\psi_j^{n+1} = \sum_{j'} (1 + \mathrm{i}\Delta t H_{\mathrm{D}})^{-1}_{jj'} \psi_{j'}^n. \tag{A.73}$$

As the matrix in brackets is tridiagonal, the inversion can be done efficiently – it requires only $\mathcal{O}(L)$ steps, i.e. of the same order as the other steps of the algorithm. The inversion method will be discussed in Appendix A8.1.

Performing the Von Neumann analysis for this method, we find

$$\xi = \frac{1}{1 + i\Delta t[(4/\Delta^2 x)\sin^2(k\Delta x/2) + V_j]}. \tag{A.74}$$

This looks more promising than the first method: for *every* choice of Δt and Δx one gets $|\xi| < 1$, so that the method is stable. It is important to take Δt small as the time-derivative of ψ_i has a first order accuracy in Δt only. The accuracy of the discretised second derivative with respect to x is still of order Δx^2.

We have neglected an important point: we would like ψ^n to be normalised at every time step. This is unfortunately not the case for the methods discussed up to now. Writing the exact solution to the continuous differential equation (A.64) as

$$\psi(x,t) = \exp(-\mathrm{i}tH/\hbar)\psi(x,0), \tag{A.75}$$

we notice that for all times t, the norm of ψ is equal to that at $t = 0$ since $\exp(-\mathrm{i}tH/\hbar)$ is a unitary operator:

$$\langle\psi(t)|\psi(t)\rangle = \langle\exp(-\mathrm{i}tH/\hbar)\psi(0)|\exp(-\mathrm{i}tH/\hbar)\psi(0)\rangle = \langle\psi(0)|\psi(0)\rangle. \tag{A.76}$$

The operator $1 \pm \mathrm{i}\Delta t H_{\mathrm{D}}$ is in general not unitary. If we are able to find a unitary operator which gives us at the same time a stable method, we have the best method we can think of (apart from finite accuracy inherent to discretisation). It turns out that the operator

$$\frac{1 - \mathrm{i}\Delta t H_{\mathrm{D}}/2}{1 + \mathrm{i}\Delta t H_{\mathrm{D}}/2} \tag{A.77}$$

does the job, and surpasses expectations: not only is it unitary and stable, it is even correct to *second* order in Δt. This can be seen by realising that using this matrix is equivalent to taking the average of H acting on ψ^n and ψ^{n+1}, or by noticing that the matrix in (A.77) is a second order approximation to $\exp(-\mathrm{i}\Delta t H/\hbar)$ in Δt.

This method is recommended for solving the time-dependent Schrödinger equation using finite difference methods. It is known as the *Crank–Nicholson* method. Note that the presence of the denominator in the operator (A.77) makes this an implicit method: a matrix inversion must be carried out at every time step, requiring $\mathcal{O}(L)$ calculations, which is the overall time scaling of the algorithm.

In the Crank–Nicholson and in the implicit scheme we need to solve a tridiagonal matrix equation. This is rather straightforward using back-substitution, and you may want to use a library routine for this purpose. However, when periodic boundaries are used, the matrix is no longer tridiagonal, but has elements in the upper right and lower left corner:

$$H = \begin{pmatrix} a_1 & b & 0 & \dots & 0 & b \\ b & a_2 & b & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & a_{N-1} & b \\ b & 0 & 0 & \dots & b & a_N \end{pmatrix}, \tag{A.78}$$

where a_k and b are complex numbers. In order to solve this equation we use the *Sherman–Morrison* formula. This is a formula which gives the inverse of a matrix of the form (in Dirac notation):

$$\mathbf{A} + |\mathbf{u}\rangle\langle\mathbf{v}|. \tag{A.79}$$

Applied to our problem, the Sherman–Morrison formula leads to a prescription for finding the solution of:

$$\mathbf{H}|\psi\rangle = (\mathbf{A} + |\mathbf{u}\rangle\langle\mathbf{v}|)|\psi\rangle = |\mathbf{b}\rangle. \tag{A.80}$$

In fact we should find solutions to the two following problems:

$$\mathbf{A}|\phi\rangle = |\mathbf{b}\rangle; \tag{A.81a}$$

$$\mathbf{A}|\chi\rangle = |\mathbf{u}\rangle. \tag{A.81b}$$

It is then easily shown that

$$|\psi\rangle = |\phi\rangle - \left(\frac{\langle\mathbf{v}|\phi\rangle}{1 + \langle\mathbf{v}|\chi\rangle}\right)|\chi\rangle \tag{A.82}$$

gives the correct solution.

In order to apply this recipe to our problem, we take

$$|\mathbf{u}\rangle = \begin{pmatrix} -a_1 \\ 0 \\ \vdots \\ 0 \\ b \end{pmatrix}; \quad |\mathbf{v}\rangle = \begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \\ (-b/a_1)^* \end{pmatrix}. \tag{A.83}$$

The complex conjugate in the last element of $|\mathbf{v}\rangle$ should not be forgotten! The matrix $\mathbf{A}$ is identical to the matrix $\mathbf{H}$ except for the upper left and lower right diagonal elements. The first should be replaced by $a_1' = 2a_1$ and the last by $a_N' = \alpha_N + b^2/a_N$. It can easily be checked that these values yield the correct matrix to appear in the

matrix equation. We need to solve two tridiagonal matrix equations for $|\phi\rangle$ and $|\chi\rangle$ to obtain the solution to the problem with periodic boundaries.

Finally, we describe a fourth method which can be applied to initial value problems: the *split operator* method. The idea of this method is based on principles which have been used extensively in Chapter 12. It consists of splitting the Hamiltonian into the kinetic and potential operator, and using representations in which these operators are diagonal. To be specific, the operator $V(x)$ is diagonal in the x-representation, whereas the kinetic energy term, $T = p^2/(2m)$ is diagonal in the p-representation. The two representations are connected through a Fourier transform:

$$\langle x|p\rangle = \frac{1}{\sqrt{2\pi}} \mathrm{e}^{\mathrm{i}px/\hbar}. \tag{A.84}$$

The time-evolution operator is split into three terms:

$$\mathrm{e}^{-\mathrm{i}\Delta t(V+T)/\hbar} = \mathrm{e}^{-\mathrm{i}\Delta tV/(2\hbar)} \mathrm{e}^{-\mathrm{i}\Delta tT/\hbar} \mathrm{e}^{-\mathrm{i}\Delta tV/(2\hbar)} + \mathcal{O}(\Delta t^3) \tag{A.85}$$

Suppose we know the initial state in the x-representation. Then we act on it with the front term (exponent of the potential) which is diagonal. Then we Fourier-transform the result and multiply it by the second term (exponent of the kinetic energy) and then, after Fourier-transforming back again, we multiply the result by the last factor (potential). This method has a similar efficiency to the Crank–Nicholson method.

Boundary value problems

We consider the Poisson equation for the potential of a charge distribution as a typical example of this category:

$$\nabla^2 \psi(\mathbf{r}) = -\rho(\mathbf{r}). \tag{A.86}$$

∇^2 is the Laplace operator and ψ is the potential resulting from the charge distribution ρ. Furthermore, there are boundary conditions, which are generally of the Von Neumann or of the Dirichlet type (according to whether the derivative or the value of the potential is given at the boundary respectively). In this section we shall discuss iterative methods for solving this type of equation. For more details the reader is referred to standard books on the subject [14, 15].

We restrict ourselves to two dimensions, and we can discretise the Laplace equation on a square lattice, analogous to the way in which this was done for the Schrödinger equation in the previous subsection:

$$\nabla^2 \psi(\mathbf{r}) = \left(\frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} \right) \psi(\mathbf{r}); \tag{A.87a}$$

$$\begin{aligned} \nabla_{\mathrm{D}}^2 \psi_{i,j} &= \frac{1}{\Delta x^2} (\psi_{i+1,j} + \psi_{i-1,j} + \psi_{i,j+1} + \psi_{i,j-1} - 4\psi_{ij}) \\ &= \nabla^2 \psi(\mathbf{r}) + \mathcal{O}(\Delta x^2). \end{aligned} \tag{A.87b}$$

This leads to the discretised form of (A.86):

$$\nabla_{\mathrm{D}}^2 \psi_{ij} = -\rho_{ij}. \tag{A.88}$$

The grid on which the Laplace operator is discretised is called a *finite difference grid*. An obvious approach to solving (A.88) is to use a relaxation method. We can interpret (A.88) as a self-consistency equation for ψ: on every grid point (i,j), the value ψ_{ij} must be equal to the average of its four neighbours, plus $\Delta x^2 \rho_{ij}/4$. So, if we start with an arbitrary trial function ψ^0, we can generate a sequence of potentials ψ^n according to

$$\psi_{i,j}^{n+1} = \frac{1}{4}(\psi_{i+1,j}^n + \psi_{i-1,j}^n + \psi_{i,j+1}^n + \psi_{i,j-1}^n) + \frac{\Delta x^2}{4}\rho_{ij}. \tag{A.89}$$

If we interpret the upper index n as the time, we recognise in this equation an initial value problem. Indeed, the stationary solutions of the initial value problem,

$$\frac{\partial \psi}{\partial t}(\mathbf{r},t) = \nabla^2 \psi(\mathbf{r},t) + \rho(\mathbf{r},t), \tag{A.90}$$

is a solution to (A.86). Equation (A.89) is the discretised version of (A.90) with $\Delta x^2 = 4\Delta t$. In fact, we have turned the boundary value problem into an initial value problem, and we are now after the stationary solution of the latter. The method given in (A.89) is called the *Jacobi method.*

Unfortunately, the relaxation to the stationary solution is very slow. For an $L \times L$ lattice with periodic boundary conditions, and taking $\rho \equiv 0$, we can find solutions similar to the Von Neumann modes of the previous subsection. These are given as:

$$\begin{aligned} &\psi(j,l;t) = \mathrm{e}^{-\alpha t}\mathrm{e}^{\pm \mathrm{i}k_x j}\mathrm{e}^{\pm \mathrm{i}k_y l}; \\ &k_x = \frac{2n}{L}\pi; \quad k_y = \frac{2m}{L}\pi; \quad n,m = 0,1,2,\ldots \\ &\mathrm{e}^{-\alpha} = [\cos(k_x) + \cos(k_y)]/2. \end{aligned} \tag{A.91}$$

Obviously, the mode with $k_x = k_y = 0$ remains constant in time, as it is a solution of the stationary equation. From the last equation we see that when k_x and k_y are not both zero, α is positive, so that stability is guaranteed. We see that the nontrivial modes with slowest relaxation (smallest α) occur for $(n,m) = (1,0)$, $(0,1)$, $(L-1,0)$, or $(0,L-1)$. For these modes, we have $\alpha = \mathcal{O}(1/L^2)$. This shows that if we discretise our problem on a finer grid in order to achieve a more accurate representation of the continuum solution, we pay a price not only via an increase in the time per iteration but also in the convergence rate, which forces us to perform more iterations before the solutions converge satisfactorily. Doubling of L causes the relaxation time to be increased by a factor of four.

We can change the convergence rate by another discretisation of the continuum initial value problem (A.90), in which instead of ψ_{ij}^{n+1} in (A.89), a linear combination of this function and the 'old one', ψ_{ij}^{n} is taken:

$$\psi_{ij}^{n+1} = (1-\omega)\psi_{ij}^{n} + \omega\left[\frac{1}{4}(\psi_{i+1,j}^{n} + \psi_{i-1,j}^{n} + \psi_{i,j+1}^{n} + \psi_{i,j-1}^{n})\right] + \frac{\Delta x^2}{4}\rho_{ij}. \tag{A.92}$$

Another way of looking at this method is to consider it as a discretisation of (A.86) with $\omega\Delta x^2 = 4\Delta t$. The solutions to this equation are again modes of the form (A.91), but now with

$$\mathrm{e}^{-\alpha} = 1 - \omega[1 - (\cos k_n + \cos k_y)/2]. \tag{A.93}$$

We see that the relaxation time of the slowest modes is still of order L^2. This method is called the *damped Jacobi method*.

Yet another type of iteration is the one in which we scan the rows of the grid in lexicographic order and overwrite the old values of the solution, ψ_{ij}^{n}, with the new ones, ψ_{ij}^{n+1}, during the scan so that these new values are then used instead of the old ones in the evaluation of the right hand side of the iteration Eq. (A.89). This method is called the *Gauss–Seidel method*. In formula this leads to the rule:

$$\psi_{i,j}^{n+1} = \frac{1}{4}(\psi_{i+1,j}^{n} + \psi_{i-1,j}^{n+1} + \psi_{i,j+1}^{n} + \psi_{i,j-1}^{n+1}) + \frac{\Delta x^2}{4}\rho_{ij}. \tag{A.94}$$

In this method, the slowest modes still relax with $\mathrm{e}^{-\alpha} \approx 1 - \mathcal{O}(1/L^2)$ although the prefactor turns out to be half that of the Jacobi method. An improvement can be achieved by considering an extension similar to the damped Jacobi method, by combining the old and new solutions according to

$$\psi_{ij}^{n+1}(\omega) = (1-\omega)\psi_{ij}^{n} + \frac{\omega}{4}[(\psi_{i+1,j}^{n} + \psi_{i-1,j}^{n+1} + \psi_{i,j+1}^{n} + \psi_{i,j-1}^{n+1})]. \tag{A.95}$$

The method can be shown to converge for $0 \leq \omega < 2$ and there exists some optimal value for ω for which the relaxation rate is given by $\mathrm{e}^{-\alpha} \approx 1 - \mathcal{O}(1/L)$, substantially better than all the methods described up to now. This optimal value turns out to be close to 2. This can be understood by a heuristic argument. In a Gauss–Seidel update of a particular site, half of its neighbours have their old, and the other half the new values. We would obviously like the new value at the present site to be equal to the average of the new values of all its neighbours. We therefore compensate for the fact that half of the neighbours still have the old value by multiplying the change in the potential ψ at each site by a factor of two. The precise optimal value for ω is related to the parameter α for the slowest mode in the simple Jacobi method, which is in general unknown (we have found this parameter above from the exact

solution which is in general unknown). In practice, a theoretical approximation of this optimal value is used, or it is determined empirically using the Jacobi method. This method is called *successive over-relaxation*.

Summarising, we can say that the iterative methods discussed in this section are subject to slow mode relaxation problems; some more than others. The reason for the slow relaxation is that the update of the value of the solution at the grid points takes only nearest neighbour points into account and acts hence on a short range. It will take many iterations in order to update a long-wavelength mode. In the next subsections we shall consider ways to get round this problem.

The conjugate gradient method for elliptic differential equations

For ease of notation and for generality we denote the PDE problem as

$$\mathbf{A}\mathbf{x} = \mathbf{b} \tag{A.96}$$

where x_{ij} is the solution; $\mathbf{x}$ and $\mathbf{b}$ are considered as *vectors* whose indices are the grid points (i, j) (in two dimensions), and $\mathbf{A}$ is a *matrix* with similar indices. When solving Poisson's equation, $\mathbf{A}$ is the matrix corresponding to the discretised Laplace operator, as defined in (A.87). Using this notation, the Jacobi relaxation method can be represented as

$$x_{ij}^{n+1} - x_{ij}^{n} = \sum_{kl} \mathbf{A}_{ij,kl} x_{kl}^{n} - b_{ij}. \tag{A.97}$$

The result on the left hand side can be viewed as the extent to which the solution $\mathbf{x}^n$ fails to satisfy the equation (A.96). The left hand side is called the *residual* $\mathbf{r}$ of the trial solution $\mathbf{x}^n$.

We would like to find the value $\mathbf{x}$ for which the residual vanishes. It turns out that this is possible using the conjugate gradients method. To this end, we consider the function

$$f(\mathbf{x}) = \tfrac{1}{2}\mathbf{x} \cdot \mathbf{A}\mathbf{x} - \mathbf{b} \cdot \mathbf{x}. \tag{A.98}$$

The gradient of this function is

$$\nabla f(\mathbf{x}) = \mathbf{A}\mathbf{x} - \mathbf{b}. \tag{A.99}$$

Therefore, when we can minimise this equation, we have solved the matrix equation (A.96). But we have already encountered a method which is very efficient at doing this: the conjugate gradient method (see Appendix A4). We now give this algorithm in pseudocode for the present problem.

$\mathbf{h} = \mathbf{b}$; ($\mathbf{b}$ is the right hand side of (A.96)
$\mathbf{g} = \mathbf{b}$;
$r_2 = \mathbf{g} \cdot \mathbf{g}$;
WHILE r_2 is not small enough DO

$\mathbf{k} = \mathbf{A}\mathbf{h}$;
$\lambda = r_2/(\mathbf{h} \cdot \mathbf{k})$;
$\mathbf{g} = \mathbf{g} - \lambda\mathbf{k}$;
$r_2' = \mathbf{g} \cdot \mathbf{g}$;
$\gamma = r_2'/r_2$;
$\mathbf{x} = \mathbf{x} + \lambda\mathbf{h}$;
$\mathbf{h} = \mathbf{g} + \gamma\mathbf{h}$;
$r_2 = r_2'$;
END WHILE.

The algorithm can be directly related to the discussion of the conjugate gradient method in Appendix A4.

The convergence of the algorithm is fast: $\mathcal{O}(L)$ steps are needed, each of which takes L^d floating point operations (as a step involves a sparse matrix–vector multiplication). We see that we only need a matrix–vector multiplication routine. The algorithm is so simple to implement and so efficient that it should always be preferred over the Gauss–Seidel method.

Fast Fourier transform methods

The first method uses the fast Fourier transform (FFT), which enables us to calculate the Fourier transform of a function defined on a one-dimensional grid of N points in a number of steps which scales as $N \log N$, in contrast with the direct method of evaluating all the Fourier sums, which takes $\mathcal{O}(N^2)$ steps. The FFT method will be described in Appendix A9. We now explain how the FFT method can be used in order to solve PDEs of the type discussed in the previous section. On a two-dimensional square grid of size $L \times L$, the FFT requires $\mathcal{O}(L^2 \log L)$ steps. In the following we assume periodic boundary conditions.

The idea behind using Fourier transforms for these equations is that the Laplace operator is diagonal in Fourier space. For our model problem of the previous section, we have

$$\sum_{jm} (\nabla_{\mathrm{D}}^2 \psi_{jm}) \mathrm{e}^{\mathrm{i}(k_x j + k_y m)} = (2\cos k_x + 2\cos k_y - 4)\hat{\psi}_{k_x k_y}, \tag{A.100}$$

where $\hat{\psi}$ is the Fourier transform of ψ. The wave numbers k_x, k_y assume the values $2\pi n/L$. We see that acting with the Laplace operator on ψ corresponds to multiplying by the factor $2\cos k_x - 2\cos k_y - 4$ in k-space. If we are to determine the solution to the PDE $\nabla_{\mathrm{D}}^2 \psi = -\rho$, we perform a Fourier transform on the left and right sides of this equation, and we arrive at

$$(4 - 2\cos k_x - 2\cos k_y)\hat{\psi}_{k_x k_y} = \hat{\rho}_{k_x k_y}. \tag{A.101}$$

This equation is easily solved because it is diagonal: we have simply L^2 independent trivial equations. The solution in direct space is found by Fourier transforming $\hat{\psi}$ back to real space.

The computational cost is that of performing the necessary Fourier transforms, and this requires $\mathcal{O}(L^2 \log L)$ floating point operations in two dimensions. The method is therefore very efficient with respect to relaxation methods, which require L^2 floating point operations per scan over the lattice. Remember this is to be multiplied by the number of iterations, which is at least $\mathcal{O}(L)$ for satisfactory accuracy (in the SOR method), so that the total time needed for these methods scales as $\mathcal{O}(L^3)$.

Multigrid methods

Multigrid methods are based on iterative ideas (see 'Boundary value problems' above) and aim – crudely speaking – at increasing the relaxation rate by updating the solution on blocks of grid points alongside the short-range update on the grid points themselves. The method is based on residual minimisation; see 'The conjugate gradient method' above. For some trial solution ψ_{ij} with residual r_{ij}, we consider the difference between ψ and the exact solution χ

$$\mathbf{A}(\psi - \chi) = r, \tag{A.102}$$

hence, if we would have a solution δ of the equation

$$\mathbf{A}\delta = r, \tag{A.103}$$

the exact solution χ would be given by

$$\chi = \psi - \delta. \tag{A.104}$$

If we have performed a few iterations, e.g. Gauss–Seidel iterations, the residual will contain on average an important contribution from the long-wavelength components and only weak short-wavelength modes, since iteration methods tend to eliminate the short-wavelength errors in the solution faster than the long-wavelength ones, because a Gauss–Seidel update involves only nearest neighbours. In the multigrid method, the remaining smooth components are dealt with in a coarser grid. We shall roughly describe the idea of the multigrid method; details will be given below. The coarse grid is a grid with half as many grid points along each cartesian direction, so in our two-dimensional case the number of points on the coarse grid is one-fourth of the number of fine grid points. A function defined on the fine grid is *restricted* to the coarser grid by a suitable coarsening procedure. We apply this coarsening procedure to the residual and perform a few Gauss–Seidel iterations on the result. The idea is that the wavelength of the long-wavelength modes is effectively twice as small on the coarse grid, so hence the long-wavelength modes can be dealt with more efficiently. The solution on the coarse grid must somehow be

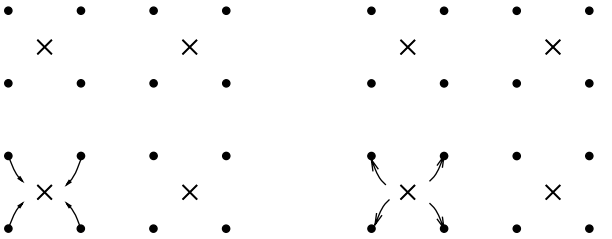


Figure A.3. Restriction and prolongation. The coarse grid points are marked by $\times$ and the fine grid points by $\bullet$. The arrows in the left hand figure represent the restriction, and those in the right hand figure, the prolongation map.

transferred to the fine grid, and the mapping used for this purpose is called *prolongation*. On the fine grid, finally, a few Gauss–Seidel iterations are then carried out to smooth out errors resulting from the somewhat crude representation of the solution at the higher level. Schematically we can represent the method described by the following diagram:

$$\begin{array}{lcl} \text{Coarse:} & & \delta = \mathbf{A}^{-1} r \to \delta \\ \text{Fine:} & \text{Gauss–Seidel} \to r \nearrow & \qquad\qquad \searrow \text{Gauss–Seidel} \end{array}$$

The obvious extension then is to play the same game at the coarse level, go to a 'doubly coarse' grid etc., and this is done in the multigrid method.

We now fill in some details. First of all, we must specifiy a *restriction mapping*, which maps a function defined on the fine grid onto the coarse grid. We consider here a coarse grid with sites being the centres of half of the plaquettes of the fine grid as in Figure A.3. The value of a function on the coarse grid point is then given as the average value of the four function values on the corners of the plaquette. Using ψ for a function on the fine grid and ϕ for a function on the coarse grid, we can write the restriction operator as

$$\phi_{ij} = (P_{l-1,l}\psi)_{ij} = \tfrac{1}{4}(\psi_{2i,2j} + \psi_{2i+1,2j} + \psi_{2i,2j+1} + \psi_{2i+1,2j+1}). \qquad \text{(A.105)}$$

ϕ_{ij} can be thought of as centred on the plaquette of the four ψ functions at the right hand side. We also need a prolongation mapping from the coarse grid to the fine grid. This consists of copying the value of the function on the coarse grid to the four neighbouring fine grid points as in the right hand side of Figure A.3. The formula for the prolongation mapping $(P_{l,l-1})$ is

$$\psi_{ij} = (P_{l,l-1}\phi)_{ij} = \phi_{i/2,j/2} \qquad \text{(A.106)}$$

where $i/2$ and $j/2$ denote (truncated) integer divisions of these indices.

We need the matrix $\mathbf{A}$ on the coarse grid. However, we know its form only on the fine grid. We use the most natural option of taking for the coarse form also a

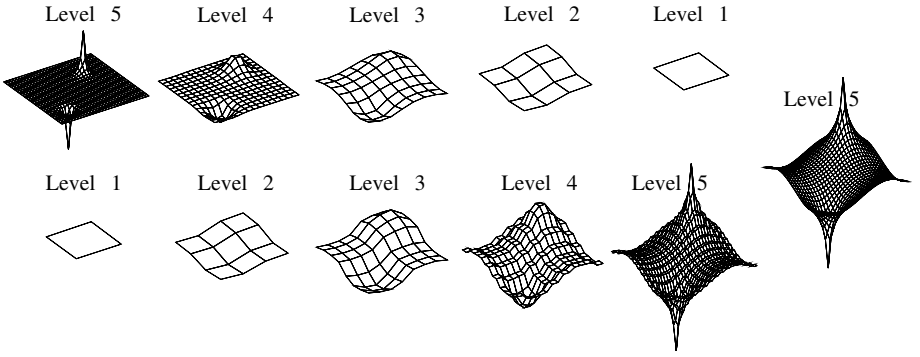


Figure A.4. The evolution of the trial solution to Poisson's equation in two dimensions with a positive and a negative point charge. The upper row shows the solution during coarsening, and the lower row during refinement. The rightmost picture is the rightmost configuration of the lower row with some extra Gauss–Seidel iterations performed.

Laplace operator coupling neighbouring sites:

$$\nabla_D^2 \phi = \frac{1}{4\Delta x^2}(\phi_{i+1,j} + \phi_{i-1,j} + \phi_{i,j+1} + \phi_{i,j-1} - 4\phi_{i,j}), \tag{A.107}$$

where Δx is taken twice as large as on the fine grid. We now have all the ingredients for the multigrid method at our disposal and we can write down the general algorithm in a recursive form:

ROUTINE MultiGrid(*Level*, ψ, *Residual*)
  Perform a few Gauss–Seidel iterations: $\psi \rightarrow \psi'$;
  IF (*Level*>0) THEN
    Calculate *Residual*;
    Restrict *Residual* to coarser grid: $\rightarrow$ *Residual*$'$;
    Set ϕ equal zero;
    Multigrid(*Level-1*, ϕ, *Residual*$'$);
  END IF;
  Prolongate $\phi :\rightarrow \phi'$;
  $\psi'' = \psi' - \phi'$;
  Perform a few Gauss–Seidel iterations: $\psi'' \rightarrow \psi'''$;
END MultiGrid.

The number of Gauss–Seidel iterations before and after the coarsening procedure must be chosen – typical values are two to five iterations. This algorithm can be coded directly (see Problem A.7). Figure A.4 shows how the method works for a two-dimensional Poisson problem with a positive and a negative charge.

How efficient is the multigrid method? The Gauss–Seidel iterations on a grid of size $L \times L$ require $\mathcal{O}(L^2)$ steps, as do the restriction and prolongation operations for such a grid. These operations must be carried out on grids of size $L^2, L^2/4, L^2/16, \ldots$, so that the total number of operations required is

$$L^2 \sum_{n=0}^{\log_2 L} \frac{1}{2^{2n}} = L^2 \frac{1}{1 - 1/4} \tag{A.108}$$

which means that a full multigrid step takes $\mathcal{O}(L^2)$ steps. For various types of problems, theorems exist which state that the number of full multigrid steps required to achieve convergence for Poisson's equation is independent of L, so that the method has the best possible time scaling, that is time $\propto L^2$.

Obviously, the definition of the coarse grid and the restriction and prolongation operators are not unique; in setting up the multigrid method there are quite a few options at each stage. We have only given one example in order to expose the method. For more details, the specialised literature should be consulted [16–19].

A8 Linear algebra problems

A8.1 Systems of linear equations

This section is devoted to an overview of matrix calculations, a topic that we have touched on several times in previous sections. A first example is the solution of a set of N linear equations of N unknowns. In a matrix formulation, the equations can be formulated as

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1N} \\ a_{21} & a_{22} & \cdots & a_{2N} \\ \vdots & \vdots & & \vdots \\ a_{N1} & a_{N2} & \cdots & a_{NN} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_N \end{pmatrix}, \tag{A.109}$$

which can be written in a more compact way as

$$\mathbf{A}\mathbf{x} = \mathbf{b}. \tag{A.110}$$

Here, $\mathbf{A}$ is a nonsingular $N \times N$ matrix with elements a_{ij}, $\mathbf{x}$ is an N-dimensional vector containing the unknowns and $\mathbf{b}$ is a known N-dimensional vector.

Dense matrices

A *dense* matrix is one whose elements are mostly nonzero, so all its elements have to be taken into account in a solution method involving such a matrix, such as the solution of a system of linear equations.

The method used for this problem is straightforward. We use the fact that the rows of **A** can be interchanged if the corresponding elements of **b** are also interchanged, and it is also allowed to replace a row of **A** by a linear combination of that row and another, provided similar transformations are performed to the elements of **b**. These properties can now be used to transform **A** into an upper triangular matrix. This is done proceeding step by step from the left column of **A** to the right one, making the elements below the diagonal for each column equal to zero. Suppose we have zeroed the lower parts of columns 1 through to $m-1$. The equation now looks as follows:

$$\begin{pmatrix} a'_{11} & a'_{12} & \cdots & a'_{1,m-1} & a'_{1m} & \cdots & a'_{1N} \\ 0 & a'_{22} & \cdots & a'_{2,m-1} & a'_{2m} & \cdots & a'_{2N} \\ \vdots & \ddots & \ddots & \vdots & \vdots & & \vdots \\ 0 & & \ddots & a'_{m-1,m-1} & a'_{m-1,m} & \cdots & a'_{mN} \\ 0 & \cdots & \cdots & 0 & a'_{mm} & \cdots & a'_{mN} \\ 0 & \cdots & \cdots & 0 & a'_{m+1,m} & \cdots & a'_{m+1,N} \\ \vdots & & & \vdots & \vdots & & \vdots \\ 0 & \cdots & \cdots & 0 & a'_{Nm} & \cdots & a'_{NN} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ \vdots \\ \cdot \\ \vdots \\ \vdots \\ x_N \end{pmatrix} = \begin{pmatrix} b'_1 \\ b'_2 \\ \vdots \\ \vdots \\ \vdots \\ \vdots \\ \vdots \\ b'_N \end{pmatrix} \tag{A.111}$$

Now we must treat column number m. To do this, we take rows $i = m+1$ to N of $\mathbf{A}'$, a'_{mi}/a'_{mm} times the mth row of $\mathbf{A}'$ and subtract, which causes all elements a'_{im}, that is, the elements of the mth column below the diagonal, to vanish.

Obviously, there is a problem when $a'_{m,m}$ is equal to zero. In that case we have to search from this diagonal element downward until we find an element $a'_{i,m}$, $i > m$, which is nonzero. We then interchange rows m and i and we proceed in the same way as described above. The calculation takes a number of m^2 steps for column m. Summing over m, we obtain a total number of $\mathcal{O}(N^3)$ steps. Unfortunately, for a generic matrix which does not contain many zeroes, the N^3 scaling cannot be altered by using other methods.

Finally, we are left with an upper triangular matrix $\mathbf{A}''$ (and a right hand side $\mathbf{b}''$) with elements a''_{ij} (b''_i):

$$\begin{pmatrix} a''_{11} & a''_{12} & \cdots & a''_{1,N-1} & a''_{1N} \\ 0 & a''_{22} & \cdots & a''_{2,N-1} & a''_{2N} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & a''_{N-1,N-1} & a''_{N-1,N} \\ 0 & 0 & \cdots & 0 & a''_{NN} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ \vdots \\ x_N \end{pmatrix} = \begin{pmatrix} b''_1 \\ b''_2 \\ \vdots \\ \vdots \\ b''_N \end{pmatrix} \tag{A.112}$$

We can now solve for the x_i straightforwardly: first, we see that its last element x_N is equal to b''_N/a''_{NN}. Then the remaining x_i are found as

$$x_i = \left(b''_i - \sum_{j=i+1}^{N} a''_{ij} x_j \right) / a''_{ii}. \tag{A.113}$$

The latter procedure is called *back-substitution*. It requires only $\mathcal{O}(N^2)$ steps since finding x_i requires $\mathcal{O}(N-i)$ steps.

For large matrices, the procedure described above is often unstable. This is because the element a'_{mm}, which is used to zero the elements below it, may become small. As the factor by which the lower rows are multiplied contains $1/a'_{mm}$, it is clear that the elements of these lower rows may become very large. Combinations of small and large numbers often cause numerical instabilities in the computer. To avoid such problems in matrix calculations it is advised to use *pivoting*. Pivoting means that the largest element (in absolute value) a'_{im}, $i = m, \ldots, N$ is looked for. If this largest element is not a'_{mm}, then rows i and m are interchanged so that the largest possible number now occurs in the denominator of the multiplication factor. This largest element is called the *pivot*.

Up to now, we have described a stable and efficient method for solving the fundamental problem of linear algebra, Eq. (A.110), finding $\mathbf{x}$ for *one* given vector $\mathbf{b}$. However, we may often have to face the problem of solving for several, or even many, vectors $\mathbf{b}_i$. For the case of several $\mathbf{b}$s (by several we mean less than the dimension of the matrix $\mathbf{A}$), we can perform the row changes needed to diagonalise $\mathbf{A}$ on the corresponding elements of all the $\mathbf{b}_i$ simultaneously. It is even possible, by initially putting the $\mathbf{b}_i$ equal to the N unit vectors, to calculate the inverse of $\mathbf{A}$. However, instead of following the procedure described above, it is preferable to zero not only the part below but also above the diagonal of $\mathbf{A}$ (meanwhile performing the same changes to the elements of the $\mathbf{b}_i$s on the right hand side) and finally to normalise the remaining diagonal elements of $\mathbf{A}$ to 1, so that it is now in unit matrix form. The $\mathbf{b}_i$ are then the columns of the inverse of $\mathbf{A}$, and no back-substitution is necessary.

For many $\mathbf{b}_i$ (i.e. more than the dimension of $\mathbf{A}$) it is tempting to use the inverse $\mathbf{A}^{-1}$ of $\mathbf{A}$ to solve for the solutions $\mathbf{x}$ by

$$\mathbf{x} = \mathbf{A}^{-1}\mathbf{b} \tag{A.114}$$

but this procedure is quite susceptible to round-off errors and therefore not recommended. A stable and efficient method for solving Eq. (A.110) for many vectors $\mathbf{b}_i$ is the *LU* decomposition. In this method, one decomposes $\mathbf{A}$ into two matrices $\mathbf{L}$ and $\mathbf{U}$ with $\mathbf{A} = \mathbf{L} \cdot \mathbf{U}$. The matrix $\mathbf{L}$ is lower, and $\mathbf{U}$ upper triangular. Because $\mathbf{L}$ and $\mathbf{U}$ contain together $N(N+1)$ nonzero elements and $\mathbf{A}$ only N^2, the problem is

redundant and we are allowed to put the diagonal elements of $\mathbf{L}$ equal to 1. In that case we have

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1N} \\ a_{21} & a_{22} & \cdots & a_{2N} \\ \vdots & \vdots & & \vdots \\ a_{N1} & a_{N2} & \cdots & a_{NN} \end{pmatrix}$$

$$= \begin{pmatrix} 1 & 0 & \cdots & 0 & 0 \\ l_{21} & 1 & \cdots & 0 & 0 \\ l_{31} & l_{32} & \cdots & 0 & 0 \\ \vdots & \vdots & & \vdots & \vdots \\ l_{N-1,1} & l_{N-1,2} & \cdots & 1 & 0 \\ l_{N1} & l_{N2} & \cdots & l_{N,N-1} & 1 \end{pmatrix} \begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1,N-1} & u_{1N} \\ 0 & u_{22} & \cdots & u_{1,N-1} & u_{2N} \\ \vdots & 0 & & \vdots & \vdots \\ \vdots & 0 & & \vdots & \vdots \\ 0 & 0 & \cdots & u_{N-1,N-1} & u_{N-1,N} \\ 0 & 0 & \cdots & 0 & u_{NN} \end{pmatrix}, \tag{A.115}$$

l_{ij} and u_{ij} being the matrix elements of $\mathbf{L}$ and $\mathbf{U}$ respectively. The decomposition matrices $\mathbf{L}$ and $\mathbf{U}$ are now easily determined: multiplying the first row of $\mathbf{L}$ with the columns of $\mathbf{U}$ and putting the result equal to a_{1i} it is found that the first row of $\mathbf{U}$ is equal to the first row of $\mathbf{A}$. We then proceed, determining l_{21} by multiplying the second row of $\mathbf{L}$ with the first column of $\mathbf{U}$, and then finding the elements of the second row of $\mathbf{U}$ and so on.

Having the LU decomposition for $\mathbf{A}$, it is easy to solve for an unknown vector $\mathbf{x}$ in the linear equation (A.110). First we search for the vector $\mathbf{y}$ obeying

$$\mathbf{L}\mathbf{y} = \mathbf{b} \tag{A.116}$$

and then for a vector $\mathbf{x}$ which satisfies

$$\mathbf{U}\mathbf{x} = \mathbf{y}. \tag{A.117}$$

Using $\mathbf{A} = \mathbf{L}\mathbf{U}$, it then immediately follows that $\mathbf{A}\mathbf{x} = \mathbf{b}$. Both problems can be solved using back-substitution, and thus require only $\mathcal{O}(N^2)$ calculations [finding $\mathbf{L}$ and $\mathbf{U}$ requires $\mathcal{O}(N^3)$ steps].

Sparse matrices

Differential operators, which frequently occur in physics, can be discretised on a finite difference grid, leading to the differential operator being represented by a *sparse* matrix: the overwhelming majority of the matrix elements are equal to zero. A typical example is the discrete Laplacian mentioned previously in the context of partial differential equations. There, space was discretised on a grid and the discrete Laplacian coupled nearest neighbour points on the grid only, so that the matrix representation of this operator contains only nonzero elements for indices

corresponding to neighbouring or identical grid points. In two dimensions with a grid constant h we obtain

$$\nabla^2\psi(\mathbf{r}) \to \frac{1}{h^2}(\psi_{i+1,j} + \psi_{i-1,j} + \psi_{i,j+1} + \psi_{i,j-1} - 4\psi_{i,j}). \tag{A.118}$$

In numerical problems involving sparse matrices, it usually pays off to exploit the sparseness: this enables us in many cases to reduce the effort involved in solving matrix problems from $\mathcal{O}(N^3)$ to $\mathcal{O}(N)$.

As an example, let us consider a tridiagonal matrix. This arises when discretising the one-dimensional Laplace operator on a grid with fixed boundary conditions. The tridiagonal matrix has the form

$$\mathbf{A} = \begin{pmatrix} a_1 & b_1 & 0 & \cdots & 0 & 0 & 0 \\ c_2 & a_2 & b_2 & \cdots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & c_{N-1} & a_{N-1} & b_{N-1} \\ 0 & 0 & 0 & \cdots & 0 & c_N & a_N \end{pmatrix}, \tag{A.119}$$

and we want to solve the equation $\mathbf{Ax} = \mathbf{d}$. This is done as in the previous subsection, by first zeroing the lower diagonal followed by backward substitution. Implementation is straightforward. Only subdiagonal elements are to be eliminated and there is only one candidate row to be subtracted off the row containing that subdiagonal element: the row just above it. All steps amount to order N.

Dedicated algorithms exist for matrices with different patterns (e.g. band-diagonal, scattered, striped) of nonzero elements. There exist also methods which rely exclusively on simple matrix operations such as a matrix–vector multiplication. You only have to supply an efficient routine for multiplying the matrix under consideration with an arbitrary vector to such a sparse matrix routine. An example is the conjugate gradient method for solving the problem $\mathbf{Ax} = \mathbf{b}$. We have encountered the conjugate gradient method for minimising an arbitrary smooth function in Appendix A4, and as a sparse-matrix method in Appendix A7.2.

A8.2 Matrix diagonalisation

We now turn to another important problem in linear algebra: that of finding the eigenvalues and eigenvectors of a matrix $\mathbf{A}$. This problem is commonly referred to as the *eigenvalue problem* for matrices. Expressing the matrix with respect to the basis formed by its eigenvectors, it assumes a diagonal form. In physics, solving the eigenvalue problem for general operators occurs frequently (solving the stationary Schrödinger equation is an eigenvalue problem for example) and in computational physics, it is usually transformed into matrix form in order to solve it efficiently in a computer.

Dense matrices: the Householder method

The majority of the matrices to be diagonalised in computational physics are Hermitian – the elements h_{ij} of an Hermitian matrix $\mathbf{H}$ satisfy $h_{ij} = h_{ji}^*$. If a physical problem leads to a non-Hermitian matrix to be diagonalised, it is always advisable to try casting the problem in a form such that the matrix becomes Hermitian, as the diagonalisation is generally more efficient for this case.

Here we restrict ourselves to the subclass of real symmetric matrices $\mathbf{S}$ because this procedure is completely analogous to that for the complex Hermitian case. Diagonalising the matrix involves finding an orthogonal matrix $\mathbf{O}$ which brings $\mathbf{S}$ to diagonal form $\mathbf{s}$:

$$\mathbf{s} = \mathbf{O}^{\mathbf{T}}\mathbf{S}\mathbf{O}. \tag{A.120}$$

The columns of $\mathbf{O}$ are the eigenvectors, which form an orthonormal set. We first construct a sequence of matrices $\mathbf{O}_i$ which, when applied one after another in a fashion as indicated by (A.120), transform $\mathbf{S}$ to a tridiagonal form. In the same spirit as the treatment of the elimination procedure described in the previous section, we assume that we have brought the matrix to tridiagonal form except for the upper left $i \times i$ block and construct the matrix $\mathbf{O}_i$ which tridiagonalises $\mathbf{S}$ one step further.

$$\mathbf{S}' = \begin{pmatrix} & & s_{jk} & & \mathbf{c} & & & \\ & & & & & & & \\ & \mathbf{c}^T & & s_{ii} & b_i & & & \\ & & & b_i & d_{i+1} & b_{i+1} & & \\ & & & & b_{i+1} & \ddots & \ddots & \\ & & & & & \ddots & \ddots & b_{N-1} \\ & & & & & & b_{N-1} & d_N \end{pmatrix} \tag{A.121}$$

The matrix $\mathbf{O}_i$ which does the job is given by

$$\mathbf{O}_i = \mathbf{I} - 2\mathbf{u}\mathbf{u}^T/(|\mathbf{u}|^2), \tag{A.122}$$

where $\mathbf{I}$ is the $N \times N$ unit matrix and $\mathbf{u}$ is the vector of size $i - 1$ given in terms of the vector $\mathbf{c}$ occurring at the right end and the bottom of the nontridiagonalised part of $\mathbf{S}'$ with zeroes at positions $i \ldots N$:

$$\mathbf{u} = \mathbf{c} - |\mathbf{c}|\mathbf{e}_{i-1}. \tag{A.123}$$

So, $\mathbf{u}\mathbf{u}^T$ is an $N \times N$ matrix, which can be written as

$$\begin{pmatrix} u_1\mathbf{u}^T & 0 & \cdots & 0 \\ u_2\mathbf{u}^T & 0 & \cdots & 0 \\ \vdots & 0 & \cdots & 0 \\ u_{i-1}\mathbf{u}^T & 0 & \cdots & 0 \\ 0\cdots 0 & 0 & \cdots & 0 \\ \vdots & \vdots & & \vdots \\ 0\cdots 0 & 0 & \cdots & 0 \end{pmatrix} \tag{A.124}$$

from which it is readily seen that multiplication of $\mathbf{S}'$ with the matrix $\mathbf{O}_i^T$ from the left, zeroes the upper $i-2$ elements of the right column $\mathbf{c}$ of the not yet diagonalised block in $\mathbf{S}'$. Similarly, multiplying the resulting matrix from the right with $\mathbf{O}_i$ zeroes the $i-2$ leftmost elements of the ith row, and we arrive at the form (A.121) but now tridiagonalised one step further.

We still have to solve the eigenvalue problem for the tridiagonalised matrix obtained in this way. This is rather difficult and we give only an outline of the procedure. The method consists again of constructing a sequence of matrices which are successive orthogonal transformations of the original matrix. After some time, the latter takes on a lower triangular form (or upper triangular, depending on the transformation) with the eigenvalues appearing on the diagonal. This algorithm is called the *QL* or *QR* algorithm according to whether one arrives at a lower or upper tridiagonal form. For tridiagonal matrices, this procedure scales as $\mathcal{O}(N^2)$.

It is clear that the tridiagonalisation procedure preceding the *QR* (or *QL*) step takes $\mathcal{O}(m^2)$ steps for stage m of the process. Therefore, the procedure as a whole takes $\mathcal{O}(N^3)$ steps. The $\mathcal{O}(N^3)$ time complexity makes the diagonalisation of large matrices very time consuming.

If the matrix is not Hermitian, the Householder method does not work. The algorithm to be used in that case is the Hessenberg method, which we shall not discuss here [1].

Sparse matrices

For diagonalisation problems, we can exploit sparseness of the matrices involved, just as in the case of linear systems of equations, discussed in Appendix A8.1. An example of a sparse diagonalisation problem is when we discretise a quantum Hamiltonian on a cubic grid, leading to a sparse matrix, as we have seen in Appendix A7.2.

A multiplication of such a matrix with a vector takes only $\mathcal{O}(N)$ steps. For sparse matrices like this, there exist special algorithms for solving the eigenvalue problem, requiring fewer steps than the Householder method, which does not exploit the

sparseness. In most cases we need only the lowest part of the eigenvalue spectrum of the discretised operator, as it is only for these eigenvalues that the discrete eigenfunctions represent the continuum solutions adequately. We can then use the *Lanczos* or *recursion* method. This method yields the lowest few eigenvalues and eigenvectors in $\mathcal{O}(N)$ steps, thus gaining two orders of efficiency with respect to Householder's method [20–22].

The Lanczos method works for Hermitian matrices, of which the Hamiltonian matrix for a quantum system is an example. We denote the matrix to be diagonalised by $\mathbf{A}$. We start with an arbitrary vector ψ_0, normalised to one and construct a second vector, ψ_1, in the following way:

$$\mathbf{A}\psi_0 = a_0\psi_0 + b_0\psi_1. \tag{A.125}$$

We require ψ_1 to be normalised and perpendicular to ψ_0; ψ_1, a_0 and b_0 are then defined uniquely:

$$\begin{aligned} a_0 &= \langle\psi_0|\mathbf{A}|\psi_0\rangle \\ b_0\psi_1 &= \mathbf{A}\psi_0 - a_0\psi_0 \\ ||\psi_1|| &= 1. \end{aligned} \tag{A.126}$$

Next we construct a normalised vector ψ_2 by letting $\mathbf{A}$ act on ψ_1:

$$\mathbf{A}\psi_1 = c_1\psi_0 + a_1\psi_1 + b_1\psi_2. \tag{A.127}$$

The vector ψ_2 is taken perpendicular to ψ_0 and ψ_1 and normalised, so ψ_2, a_1, b_1 and c_1 are determined uniquely too. We proceed in the same way, and at step p we have

$$\mathbf{A}\psi_p = \sum_{q=0}^{p-1} c_p^{(q)}\psi_q + a_p\psi_p + b_p\psi_{p+1} \tag{A.128}$$

where ψ_{p+1} is taken orthogonal to all its predecessors ψ_q, $q \leq p$. Using the fact that $\mathbf{A}$ is Hermitian we find for $q < p-1$:

$$c_p^{(q)} = \langle\psi_q|\mathbf{A}\psi_p\rangle = \langle\mathbf{A}\psi_q|\psi_p\rangle = 0. \tag{A.129}$$

The last inequality follows from the fact that we have encountered q already in the iteration procedure, so ψ_p is perpendicular to $\mathbf{A}\psi_q$. In the same way we find for $c_p^{(p-1)}$:

$$c_p^{(p-1)} = \langle\psi_{p-1}|\mathbf{A}\psi_p\rangle = \langle\mathbf{A}\psi_{p-1}|\psi_p\rangle = b_{p-1}. \tag{A.130}$$

So after p steps we obtain

$$\mathbf{A}\psi_p = b_{p-1}\psi_{p-1} + a_p\psi_p + b_p\psi_{p+1}, \tag{A.131}$$

where ψ_{p+1} is normalised and orthogonal to ψ_p and ψ_{p-1}. By the foregoing analysis, it is then orthogonal to all the previous ψ_q ($q < p-1$). We see that $\mathbf{A}$, expressed

with respect to the basis ψ_p, takes on a tridiagonal form; the eigenvalues of the $p \times p$ tridiagonal matrix obtained after p Lanczos-iterations will converge to the eigenvalues of the original matrix. It can be shown that the lowest and highest eigenvalues converge most rapidly in this process, so that the method can be used successfully for these parts of the spectrum. The number of steps needed to achieve sufficient accuracy depends strongly on the spectrum; for example, if a set of small eigenvalues is separated from the rest by a relatively large gap, convergence to these small eigenvalues is fast.

A9 The fast Fourier transform

A9.1 General considerations

Fourier transforms occur very often in physics. They can be used to diagonalise operators, for example when an equation contains the operator ∇^2 acting on a function ψ. After Fourier-transforming the equation, this differential operator becomes a multiplicative factor $-k^2$ in front of the Fourier transform of ψ, where $\mathbf{k}$ is the wave vector. In quantum mechanics the stationary solutions for a free particle are found in this way: these are plane waves with energy $E = \hbar^2 k^2/2m$.

In data analysis, the Fourier transform is often used as a tool to reduce the data: in music notation one uses the fact that specifying tones by their pitch (which is nothing but a frequency) requires much less data than specifying the real-time oscillatory signal. A further application of Fourier transform is filtering out high-frequency noise present in experimental data.

The fast Fourier transform, or FFT, is a method to perform the Fourier transform much more efficiently than the straightforward calculation. It is based on the Danielson–Lanczos lemma (see the next subsection) and uses the fact that in determining the Fourier transform of a discrete periodic function, the factor $\mathrm{e}^{\mathrm{i}kx}$ assumes the same value for different combinations of x and k. We shall explain the method in the next subsection, emphasising the main idea and why it works, but avoid the details involved in coding it up for the computer. Here we recall the definition and some generalities concerning the Fourier transform.

First, we define the Fourier transform for the one-dimensional case. Generalisation to more dimensions is obvious. We consider first the Fourier transform of a periodic function $f(x)$ with period L, defined on a discrete set of N equidistant points with spacing $h = L/N$:

$$x_j = jL/N, \quad j = 0, \ldots, N-1. \tag{A.132}$$

Note that in contrast to our discussion of partial differential equations, L does not denote the number of grid points (N), but a distance. In that case, the Fourier

transform $\hat{f}$ is also a function defined on discrete points, k_n, defined by

$$k_n = \frac{2n\pi}{L}, \quad n = 0, \ldots, N-1. \tag{A.133}$$

The Fourier transform reads

$$\hat{f}(k_n) = \sum_{j=0}^{N-1} f(x_j) \mathrm{e}^{2\pi \mathrm{i} nj/N}. \tag{A.134}$$

We can also define the backward transform:

$$f(x_j) = \frac{1}{L} \sum_{j} \hat{f}(k_n) \mathrm{e}^{-2\pi \mathrm{i} nj/N}. \tag{A.135}$$

If the points are not discrete, the Fourier transform is defined for the infinite set of k-values $2\pi n/L$ with $n = 0, \ldots \infty$, and the sum over the index j is replaced by an integral:

$$\hat{f}(k_n) = \int_0^L \mathrm{d}x f(x) \mathrm{e}^{\mathrm{i}k_n x}. \tag{A.136}$$

If the function is not periodic, L goes to infinity, and the Fourier transform is defined for every real k as

$$\hat{f}(k) = \int_{-\infty}^{\infty} \mathrm{d}x f(x) \mathrm{e}^{\mathrm{i}kx}, \tag{A.137}$$

with backward transform

$$f(x) = \frac{1}{2\pi} \int_{-\infty}^{\infty} \mathrm{d}k \hat{f}(k) \mathrm{e}^{-\mathrm{i}kx}. \tag{A.138}$$

When Fourier-transforming a nonperiodic function on a computer, it is usually restricted to a finite interval using discrete points within that interval. Furthermore, the function is considered to be continued periodically outside this interval. This restriction to a discrete set combined with periodic continuation should always be handled carefully. For example, after Fourier-transforming and back again, we must use the result only on the original interval: extending the solution outside this interval leads to periodic continuation of our function there. The usual problems involved in discretisation should be taken care of – see Appendix A5.

A9.2 How does the FFT work?

To determine the discrete Fourier transform directly, the sum over j is carried out for each k_n, that is, N times a sum of N terms has to be calculated, which amounts to a total number of multiplications/additions of N^2. In the fast Fourier transform, it is possible to reduce the work to $\mathcal{O}(N \log_2 N)$ operations. This is an important difference: for large N, $\log_2 N$ is much smaller than N. For example, $\log_2 10^6 \approx 20$!

From now on, we shall use the notation $f(x_j) = f_j$ and $\hat{f}(x_n) = \hat{f}_n$. We assume that the number of points N is equal to a power of 2: $N = 2^m$. If this is not the case, we put the function for j-values from N upward to the nearest power of 2 to zero and do the transform on this larger interval. The Danielson–Lanczos lemma is simple. The points x_j fall into two subsets: one with j even and one with j odd. The Fourier transforms of these subsets are called $\hat{f}_{2|0}$ and $\hat{f}_{2|1}$ This notation indicates that we take the indices j modulo 2 and check if the result is equal to 0 (j even) or 1 (j odd). We obtain

$$\begin{aligned}\hat{f}_n &= \sum_{j=0}^{N-1} f_j \mathrm{e}^{2\pi \mathrm{i} nj/N} \\ &= \sum_{j=0}^{N/2-1} f_{2j} \mathrm{e}^{4\pi \mathrm{i} nj/N} + \mathrm{e}^{2\pi \mathrm{i} n/N} \sum_{j=0}^{N/2-1} f_{2j+1} \mathrm{e}^{4\pi \mathrm{i} nj/N} \\ &= \hat{f}_{2|0} + \mathrm{e}^{2\pi n/N} \hat{f}_{2|1}. \end{aligned} \tag{A.139}$$

The Fourier transform on the full set of N points is thus split into two transforms on sets containing half the number of points. If we were to calculate these sub-transforms using the direct method, each would take $(N/2)^2$ steps. Adding them as in the last line of (A.139) takes another N steps, so in total we have $N^2/2 + N$ as opposed to N^2 operations if we had applied the direct method to the original series. Where does the gain in speed come from? Compare the transforms for k_0 and $k_{N/2}$ respectively. For any point x_j with j even, we have the exponential $\exp(2\pi \mathrm{i} nj/N)$ which are equal for these two k-values, but in the direct method, these exponentials are calculated twice! The same holds for odd j-values and for other pairs of k_n-values spaced by $k_{N/2}$. Indeed, the two sub-transforms $\hat{f}_{2|0}$ and $\hat{f}_{2|1}$ are periodic with period $N/2$ instead of N – therefore their evaluation for all $j = 0, \ldots, N-1$ takes only $(N/2)^2$ steps, and the total work involved in finding the Fourier transform is reduced approximately by a factor of two. That is not yet the $N \log_2 N$ promised above. But this can be found straightforwardly by applying the transform of (A.139) to the sub-transforms of length $N/2$. One then splits the sum into four sub-transforms of length $N/4$. The reader can verify that the new form becomes

$$\hat{f}_n = \hat{f}_{4|0} + \mathrm{e}^{4\pi \mathrm{i} k_n/N} \hat{f}_{4|2} + \mathrm{e}^{2\pi \mathrm{i} k_n/N} \hat{f}_{4|1} + \mathrm{e}^{6\pi \mathrm{i} k_n/N} \hat{f}_{4|3}. \tag{A.140}$$

The same transformation can be performed recursively m times (remember $N = 2^m$) to arrive at $N \log_2 N$ sub-transforms of length 1, which are trivial.

It will be clear that the FFT can be coded most easily using recursive programming. When recursion is avoided for reasons of efficiency, a bit more bookkeeping is required: see Refs. [1, 23, 24].

Exercises

A.1 [C] Write routines for generating the spherical Bessel functions j_l and n_l according to the method described in Appendix A2. For j_l, downward recursion is necessary to obtain accurate results.

The upper value $L_{\max}$ from which the downward recursion must start must be sufficiently large. In this problem, we take the following value for N:

$$L_{\max} = \max\left\{\left[\frac{3[x]}{2}\right] + 20, l + 20\right\}$$

where $[x]$ is the largest integer smaller than x and l is the value we want the spherical Bessel function for.

Write a function that yields $j_l(x)$. Check the result using the following values:

$$j_5(1.5) = 6.696\,205\,96 \cdot 10^{-4}$$

$$n_5(1.5) = -94.236\,110\,1.$$

A.2 Check the statements (i)–(v) given on page 565. In all cases, proofs by induction can be given. The following hints may be useful:

(i) Use (A.22a) and (A.23a).
(ii) Use (A.22a), (A.22b) and (A.23b).
(iii) Use (A.22a) and (A.22b).

The last two items are proven together. Equations (A.22a) and (A.22b), and statement (i) are used in the proof.

A.3 [C] Consider the Verlet algorithm:

$$x_{n+1} = 2x_n - x_{n-1} + h^2 F[x_n, n] + \mathcal{O}(h^4)$$

where $x_n = x(t + nh)$. The $\mathcal{O}(h^4)$ means that the deviation of the exact solution from the numerical one is smaller than $\alpha_n h^4$, where α_n are finite numbers that are nonvanishing for $h \to 0$. We write the exact solution of the differential equation at times nh as $\tilde{x}_n$, and the numerical solution x_n deviates from the latter by an amount δ_n:

$$x_n = \tilde{x}_n + \delta_n.$$

(a) Write down an equation for δ_n and show that integration over an interval of finite width using a number of steps of size h, yields an error in the final result which, in the absence of divergences in the force (or the solution), is at most $\mathcal{O}(h^2)$.
(b) Carry out a similar analysis for the Numerov algorithm, showing that the final accuracy is at most $\mathcal{O}(h^4)$.
(c) Discuss when the 'worst case' error ($\mathcal{O}(h^2)$ for Verlet and $\mathcal{O}(h^4)$ for Numerov) is found.
(d) [C] Check the results of (a), (b) and (c) by writing routines for the two algorithms.

A.4 [C] Consider the Schrödinger equation in one dimension for a particle moving in the *Morse potential*:

$$\left[-\frac{\mathrm{d}^2}{\mathrm{d}x^2} + V_0(\mathrm{e}^{-2x} - 2\mathrm{e}^{-x})\right]\psi(x) = E\psi(x).$$

We shall take $\hbar^2/2m = 1$ in the following. This equation can be solved analytically (it can be mapped onto the Laplace equation which plays an important role in the solution of the hydrogen atom [25]), the bound state energies are given by

$$E_n = -V_0 \left(1 - \frac{n + \frac{1}{2}}{\sqrt{V_0}}\right), \quad n = 0, 1, 2, \ldots$$

In this problem it is assumed that you have a routine available for integrating the one-dimensional Schrödinger equation: you can write a Numerov routine or use a library routine. We want to determine the bound state spectrum numerically. This is done in the following way. First, a range $x_{\max}$ is defined, beyond which we can safely approximate the potential by $V(x) = 0$; $x_{\max} \approx 10$ is a good value for this range. For some energy E, the Numerov routine can be used to integrate the Schrödinger equation numerically up to the range $x_{\max}$ yielding a solution $u(x)$, and beyond $x_{\max}$, the solution, which we denote as $v(x)$, is known analytically:

$$v(x) = A\mathrm{e}^{-qx} + B\mathrm{e}^{qx},$$
$$q = \sqrt{-E}.$$

Consider the *Wronskian* W:

$$W = u'(x_{\max})v(x_{\max}) - u(x_{\max})v'(x_{\max})$$

(the prime $'$ denotes a derivative). For a bound state, the coefficient B in the solution v should vanish and we have for the Wronskian:

$$W_{B=0} = [u'(x_{\max}) + qu(x_{\max})]\mathrm{e}^{-qx_{\max}}.$$

The matching condition between the analytical solution v and the numerical one (u), is equivalent to the Wronskian becoming equal to zero, and we thus have to find the energies for which the Wronskian with $B = 0$ vanishes. These energies can be found using, for example, the secant method.

(a) [C] Write a function which uses the Numerov procedure for solving the Schrödinger equation and returns the Wronskian with $B = 0$ as a function of the energy E.

(b) [C] Write a code for the secant method of Appendix A3. Test this with some simple function you know the roots of, for example $\sin x$.

(c) [C] Use the secant code for finding the energies for which the Wronskian with $B = 0$ vanishes. Note that the energies lie between 0 and $-V_0$. The search starts from $E = -V_0$, and after increasing the energy by a small step (which is predefined in the program or by the user), it is checked if the Wronskian changes sign. If this is the case, the secant method is executed for the last energy interval. When the root is found, the energy is increased again until the Wronskian changes sign. The procedure is repeated until the energy becomes positive.

Check if the energies match the exact values given above.

A.5 Starting from Newton's interpolation formula, derive the Gauss interpolation formula to order three:

$$x_{i+t} = x_i + t\delta x_{i+1/2} + \frac{t(t-1)}{2!}\delta^2 x_i + \frac{t(t^2-1)}{3!}\delta^3 x_{i+1/2} + \cdots$$

A.6 (a) Show explicitly that equating the time derivative $(\psi_i^{n+1} - \psi_i^n)/\Delta t$ to the average value of the Hamiltonian acting on ψ_i^{n+1} and on ψ_i^n (see Eqs. (A.67), (A.68)), yields the Crank–Nicholson algorithm.

(i) Show that the operator

$$\frac{1 - \frac{1}{2}\mathrm{i}\Delta t H}{1 + \frac{1}{2}\mathrm{i}\Delta t H}$$

is a second order approximation in Δt to $\exp(-\mathrm{i}\Delta t H)$.

A.7 [C] Consider Poisson's equation for two opposite point charges on a square of linear size Lh (see Figure A.4). We take L to be a multiple of 4. The charges are placed at positions $\mathbf{r}_{1/4,1/4} = Lh(1/4, 1/4)$ and $\mathbf{r}_{3/4,3/4} = Lh(3/4, 3/4)$, so that the charge distribution is given as

$$\rho(\mathbf{r}) = \delta(\mathbf{r} - \mathbf{r}_{1/4,1/4}) - \delta(\mathbf{r} - \mathbf{r}_{3/4,3/4}).$$

We discretise Poisson's equation on an $L \times L$ grid with periodic boundary conditions, and with grid constant h. The Laplace operator is given in discretised form in Appendix A7.2 ('Boundary value problems') and the discretised charge distribution is given in terms of Kronecker deltas:

$$\rho(i,j) = (\delta_{i,L/4}\delta_{j,L/4} - \delta_{i,3L/4}\delta_{j,3L/4})/h^2.$$

As the Laplace operator and the charge distribution both contain a pre-factor $1/h^2$, this drops out of the equation.

(a) [C] Solve Poisson's equation (A.88) for this charge distribution using the Gauss–Seidel iteration method.

(b) [C] Apply the multigrid method using the prolongation and discretisation mappings described in Appendix A7.2 ('Multigrid methods') to solve the same problem.

(c) [C] Compare the performance (measured as a time to arrive at the solution within some accuracy) for the methods in (a) and (b). Check in particular that the number of multigrid steps needed to obtain convergence is more or less independent of L.

References

[1] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes*, 2nd edn. Cambridge, Cambridge University Press, 1992.

[2] R. W. Hamming, *Numerical Methods for Scientists and Engineers*. International Series in Pure and Applied Mathematics, New York, McGraw-Hill, 1973.

[3] G. E. Forsythe, M. A. Malcolm, and C. B. Moler, *Computer Methods for Mathematical Computations*. Englewood Cliffs, NJ, Prentice-Hall, 1977.

[4] J. Stoer and R. Bulirsch, *Introduction to Numerical Analysis*. New York, Springer Verlag, 1977.
[5] D. Kincaid and W. Cheney, *Numerical Analysis*. Belmont, CA, Wadsworth, 1991.
[6] D. Greenspan and V. Casulli, *Numerical Analysis for Applied Mathematics, Science and Engineering*. Redwood City, CA, Addison-Wesley, 1988.
[7] R. P. Brent, *Algorithms for Minimization without Derivatives*. Englewood Cliffs, NJ, Prentice-Hall, 1973.
[8] D. Levesque and L. Verlet, 'Molecular dynamics and time reversibility,' *J. Stat. Phys.*, **72** (1993), 519–37.
[9] J. Delambre, *Mem. Acad. Turin*, **5** (1790), 1411.
[10] B. Numerov, *Publ. l'Observ. Astrophys. Central Russie*, **2** (1933), 188.
[11] D. R. Hartree, *Numerical Analysis*, 2nd edn. London, Oxford University Press, 1958.
[12] L. M. Milne-Thomson, *The Calculus of Finite Differences*. London, Macmillan, 1960.
[13] S. Herrick, *Astrodynamics*, vol. 2. New York, Van Nostrand Reinhold, 1972.
[14] R. S. Varga, *Matrix Iterative Analysis*. Englewood Cliffs, NJ, Prentice-Hall, 1962.
[15] D. M. Young, *Iterative Solution of Large Linear Systems*. New York, Academic Press, 1971.
[16] W. Hackbusch and U. Trottenberg, eds., *Multigrid Methods*, Berlin, Springer, 1982.
[17] W. Hackbusch, *Multigrid Methods and Applications*. Berlin, Springer, 1984.
[18] P. Wesseling, *An Introduction to Multigrid Methods*. Chichester, John Wiley, 1992.
[19] W. L. Briggs, *A Multigrid Tutorial*. Philadelphia, SIAM, 1987.
[20] H. H. Roomany, H. W. Wyld, and L. E. Holloway, 'New method for the Hamiltonian formulation for lattice spin systems,' *Phys. Rev. D*, **21** (1980), 1557–63.
[21] R. Haydock, V. Heine, and M. J. Kelly, 'Electronic structure based on the local atomic environment for tight-binding bands,' *J. Phys. C*, **5** (1972), 2845–58.
[22] C. G. Paige,, 'Computational variants of the Lanczos method for the eigenproblem,' *J. Inst. Math. Appl.*, **10** (1972), 373–81.
[23] J. W. Cooley and J. W. Tukey, 'An algorithm for the machine calculation of complex Fourier series,' *Math. Comp.*, **19** (1965), 297–301.
[24] R. J. Higgins, 'Fast Fourier transform: an introduction with some minicomputer experiments,' *Am. J. Phys.*, **44** (1976), 766–73.
[25] A. Messiah, *Quantum Mechanics*, vols. 1 and 2. Amsterdam, North-Holland, 1961.

Appendix B

Random number generators

B1 Random numbers and pseudo-random numbers

Random numbers are used in many simulations, not only of gambling tables but also of particle accelerators, fluids and gases, surface phenomena, traffic and so forth. In all these simulations some part of the system responsible for the behaviour under investigation is replaced by events generated by a random number generator, such as particles being injected into the system, whereas the source itself is not considered. Here we discuss various methods used for generating random numbers and study the properties of these numbers.

Random numbers are characterised by the fact that their value cannot be predicted. More precisely, if we construct a sequence of random numbers, the probability distribution for a new number is independent of all the numbers generated so far. As an example, one may think of throwing a die: the probability of throwing a 3 is independent of the results obtained before. Pure random numbers may occur in experiments: for a radioactive nucleus having a certain probability of decay, it is not possible to predict *when* it will decay. There is an internet service, `http://www.fourmilab.ch/hotbits/` which creates random numbers in this way and sends them over the internet (a careful correction has been carried out to remove any bias resulting in a majority of either 1 or 0). These numbers are *truly* random [1, 2].

On the other hand, random numbers as generated by a computer are not truly random. In all computer generators the new numbers are generated from the previous ones by a mathematical formula. This means that the new value is fully determined by the previous ones! However, the numbers generated with these algorithms often have properties making them very suitable for simulations. Therefore they are called *pseudo-random* numbers. In fact, a 'good' random number generator yields sequences of numbers that are difficult to distinguish from sequences of pure random numbers, although it is not possible to generate pseudo-random numbers that are completely indistinguishable from pure ones. In the following, we drop the prefix 'pseudo' when we are dealing with random numbers generated in a computer.

Random numbers from standard generators are uniformly distributed over the interval [0, 1]. This means that each number between 0 and 1 has the same probability of occurrence, although in reality only values on a dense discrete set are possible because of the finite number of bits used in the representation of the numbers. It is also possible to make nonuniform random number generators: in that case, some numbers have a higher probability of occurrence than others (nonuniform generators will be considered in Appendix B3). We define a distribution function P such that $P(x)\mathrm{d}x$ gives us the probability of finding a random number in the interval $(x, x + \mathrm{d}x)$. For the uniform distribution we have

$$P(x) = \begin{cases} 1 & \text{for } 0 \leq x \leq 1 \\ 0 & \text{else.} \end{cases}$$

A more precise criterion for the quality of random number generators involves the absence of correlations. The absence of correlations can be expressed in terms of more complicated distribution functions. As an example, we would like the distribution function $P(x_i, x_{i+1})$, giving the probability for the successive occurrence of two random numbers x_i and x_{i+1}, to satisfy

$$P(x_i, x_{i+1}) = P(x_i)P(x_{i+1}) = 1. \tag{B.1}$$

Similar conditions are required for $P(x_i, x_j)$ with $j - i > 1$ and also for distribution functions of more than two variables.

B2 Random number generators and properties of pseudo-random numbers

In a computer, (pseudo-)random numbers are always represented by a finite number of bits. This means that they can conveniently be interpreted as integers. If these integer numbers are distributed uniformly, we can obtain real numbers by dividing them by the largest integer that can be generated.

As a first example, we consider the most commonly used generator, the *linear congruent* or *modulo* generator [3]. Given fixed integers a, c and m, we have the following prescription for generating the next random integer x_i from the previous one (x_{i-1}):

$$x_i = (a \cdot x_{i-1} + c) \bmod m, i > 0. \tag{B.2}$$

The initial number x_0, which must be chosen before starting the process of generating numbers, is called the *seed* of the generator. A real number between 0 and 1 is obtained by dividing x_i by m. In most cases c is taken to be equal to 0. We now see a problem arising when $x_0 = 0$: in that case, all subsequent numbers remain equal to 0, which can hardly be taken for a random sequence. The choice for the seed must therefore be made carefully. Since $x = 0$ is ruled out, the maximum number of

different random numbers is $m-1$. It turns out that there exist special combinations of a and m allowing for $m-1$ different random numbers to be generated. If the first number of the sequence reappears, the sequence will repeat itself and the random character is obviously lost.

To show that the choice of a and m is indeed a subtle one, consider $a = 12$ and $m = 143$. In that case:

$$\begin{aligned} x_{i+1} &= 12x_i \bmod 143 \\ x_{i+2} &= 12^2 x_i \bmod 143 \\ &= 144x_i \bmod 143 = x_i \end{aligned} \tag{B.3}$$

so the sequence has a length of only 2! It is not too difficult to see that the following conditions are necessary and sufficient for obtaining a maximum length of the random number sequence:

- x_0 is relatively prime to m, that is, x_0 and m have no common factors;
- a is a primitive element modulo m, that is, it is the integer with the largest *order modulo m* possible. The order modulo m, denoted by λ, for a number a is the smallest integer number λ for which it holds that $a^\lambda \mod m = 1$.

The maximum length of the random number sequence is λ for the primitive element a. If m is prime, this length is equal to $m-1$. For $m = 2^r$, the maximum length is 2^{r-1}.

If we use r bits to represent the random numbers of our generator, there are in principle 2^r different numbers possible. The choice $m = 2^r$ is a convenient one since calculations involving computer words are carried out modulo 2^r after cutting off beyond the r least significant bits. In particular, for 32-bit integers with the first bit acting as a sign-bit, r can be chosen as 31. Primitive elements for this value of m are all integers a with $a \mod 8 = 3$ or 5. A random generator which has been used frequently is IBM's `randu` which used $m = 2^{31}$ and $a = 65\,539$. This turns out to have poor statistical properties (see below), and moreover it is not really portable as it depends on the computer's word length and on a specific handling of the overflow in integer multiplication. A better choice is $m = 2^{31} - 1$, which is a prime, leading to a maximal sequence length of $2^{31} - 2 \approx 2 \times 10^9$. A primitive element for this m is $a = 16\,807$ [4]. Schrage has given a method for calculating $x_i \cdot a \bmod m$ without causing overflow, even if $x_i \cdot a$ is larger than the computer word size [5, 6]; see also Problem B.1. In fact, 16 807 is not the only primitive element modulo $2^{31} - 1$: there are more than half a million of them! Extensive research has been carried out to find the 'best' ones among them and those to which Schrage's method is applicable [7]. It turns out that 16 807 is not the very best, but it is not bad at all. Moreover, people feel safe using it, because it has been tested more extensively than any other multiplier and has not exhibited really dangerous

behaviour in any test so far [4]. The sequence lengths in all these examples might seem large, but in practice they are not sufficient for large-scale simulations, so that sometimes one has to look for generators with larger periods.

Before treating another type of random number generator, we discuss statistical deficiencies intrinsic to all types of generators, although some suffer more from them than others. It turns out that when pairs of subsequent random numbers from a sequence are considered, fairly small correlations are found between them. However, if triples or higher multiples of subsequent random numbers are taken, the correlations become stronger. This can be shown by taking triples of random numbers, for example, and considering these as the indices of points in three-dimensional space. For pure random numbers, these points should fill the unit cube homogeneously, but for pseudo-random sequences from a modulo generator, the points fall near a set of parallel planes. A theoretical upper bound to the number of such planes in dimension d has been found [8] – it is given by $(d!m)^{1/d}$. In general one can say that the better the random number generator, the more planes fill the d-dimensional hypercube. It is not clear to what extent such deviations from pure random sequences influence the outcome of simulations using random number generators: this depends on the type of simulation.

In the case of the modulo random number generator, the importance of correlations varies with the multiplier a. A small value of the multiplier a for example results in a small random number x_i to be followed by a few more relatively small numbers, and this is of course highly correlated. To obtain a good multiplier, extensive tests have to be performed [4, 7].

It should be stressed that bad random number generators abound in currently available software, and it is recommended to check the results of any simulation with different random number generators. Surprisingly, random generators may be better on paper than others but have less favourable properties in particular simulations. It has been noted, for example, that the simple modulo generator has better properties in cluster Monte Carlo simulations (see Section 15.5.1) than several others that perform better in general statistical tests [9], because the formation of the clusters induces a higher sensitivity to long-range correlations.

A second example of a random number generator, which suffers less from correlations than the modulo generator, is a shift-register generator. This works as follows. The random numbers are strings of, say, r bits. Each bit of the next number in the sequence is determined by the corresponding bits of the previous n numbers. If we denote the kth bit of the ith number in the sequence by $b_i^{(k)}$, we can write down the production rule for the new bits:

$$b_i^{(k)} = (c_1 b_{i-1}^{(k)} + c_2 b_{i-2}^{(k)} + \cdots + c_n b_{i-n}^{(k)}) \bmod 2 \tag{B.4}$$

where the numbers c_i are all equal to 0 or 1. We see that the shift-register generator is a generalisation of the modulo generator. The maximum cycle length is $2^n - 1$. This maximum length is obtained for special combinations of the numbers c_i. Of course, the algorithm presented can also be used to generate rows of random bits [10].

A simple form of this generator is the one for which only two c_i are nonzero:

$$x_i^{(k)} = \left(x_{i-q}^{(k)} + x_{i-p}^{(k)}\right) \bmod 2. \tag{B.5}$$

The sum combined with the mod 2 condition is precisely the XOR operation which can be executed very fast in a computer. 'Magical' (p, q) pairs yielding an optimal cycle length are: $(98, 27)$, $(521, 32)$ and finally $(250, 103)$, which is frequently used.

Before the generator (B.4) can be started, the first n random numbers have to be known already. These can be generated with a modulo generator. Since only a limited number of starting values is needed, correlations between these are not yet noticeable [11].

B3 Nonuniform random number generators

Random numbers with a nonuniform distribution are usually constructed starting from a uniform generator. In this section, we show how this can be done. As a first example we consider a generator with a Gaussian distribution:

$$P(x) = \mathrm{e}^{-(x-x_0)^2/2\sigma^2}. \tag{B.6}$$

From the central limit theorem, which states that the sum of many uncorrelated random numbers is distributed according to a Gaussian with a width proportional to $1/\sqrt{N}$, it follows directly that we can obtain a Gaussian distribution just by adding n uniform random numbers; the higher n the more accurate this distribution will match the Gaussian form. If we want to have a Gaussian with a width σ and a centre $\bar{x}$, we transform the sum S of n uniform random numbers according to

$$x = \bar{x} - 2\sigma(S/n - 1/2)\sqrt{3} \tag{B.7}$$

This method for achieving a Gaussian distribution of the random numbers is very inefficient as we have to generate n uniform random numbers to obtain a single Gaussian one. We discuss more efficient methods below.

More generally, one can make a nonuniform random number generator using a real function f, and for each number x generated by a uniform generator, taking $f(x)$ as the new nonuniform random number, where f is a function designed such as to arrive at the prescribed distribution P. As the number of random numbers lying between x and $x + \mathrm{d}x$ is proportional to $\mathrm{d}x$ and the same number of nonuniform random numbers $y = f(x)$ will lie between y and $y + \mathrm{d}y$ with $\mathrm{d}y/\mathrm{d}x = f'(x)$, the

density of the numbers $y = f(x)$ is given by $1/f'(x)$, so this should be equal to the prescribed distribution $P(y)$:

$$1/f'(x) = P(y) \text{ with} \tag{B.8a}$$

$$y = f(x). \tag{B.8b}$$

We must construct a function f that yields the prescribed distribution P, i.e. one that satisfies (B.8b). To this end, we use the following relation between f and its inverse f^{-1}:

$$(f^{-1})'(y) f'(x) = 1 \tag{B.9}$$

from which it follows that

$$P(y) = (f^{-1})'(y). \tag{B.10}$$

There is a restricted number of distributions for which such a function f can be found, because it is not always possible to find an invertible primitive function to the distribution P. A good example for which this *is* possible is the Maxwell distribution for the velocities in two dimensions. Taking the Boltzmann factor $1/(k_B T)$ equal to $1/2$ for simplicity, the velocities are distributed according to

$$P(v_x, v_y)\mathrm{d}v_x\mathrm{d}v_y = \mathrm{e}^{-v^2/2}\mathrm{d}v_x\mathrm{d}v_y = \mathrm{e}^{-v^2/2} v\mathrm{d}v\mathrm{d}\varphi = P(v)\mathrm{d}v\mathrm{d}\varphi, \tag{B.11}$$

so the norm v of the velocity is distributed according to

$$P(y) = y\mathrm{e}^{-y^2/2}. \tag{B.12}$$

From (B.10) we find that the function f is defined by

$$f^{-1}(y) = -\mathrm{e}^{-y^2/2} + \text{Const.} = x \tag{B.13}$$

so that

$$y = f(x) = \sqrt{-2\ln(\text{Const.} - x)}. \tag{B.14}$$

Because x lies between 0 and 1, and y between 0 and ∞, we find for the constant the value 1, and a substitution $x \to 1 - x$ (preserving the interval [0,1] of allowed values for x) enables us to write

$$y = f(x) = \sqrt{-2\ln(x)}. \tag{B.15}$$

This method is very efficient since each random number generated by the uniform generator yields a nonuniformly distributed random number.

From (B.11), we see that it is possible to generate Gaussian random numbers starting from a distribution (B.12), since we can consider the Maxwell distribution as a distribution for the generation of two independent Gaussian random numbers $v_x = v\cos\varphi$, $v_y = v\sin\varphi$. From this it is clear that by generating two random numbers, one being the value v with a distribution according to (B.12) and another being the value φ with a uniform distribution between 0 and 2π, we can construct

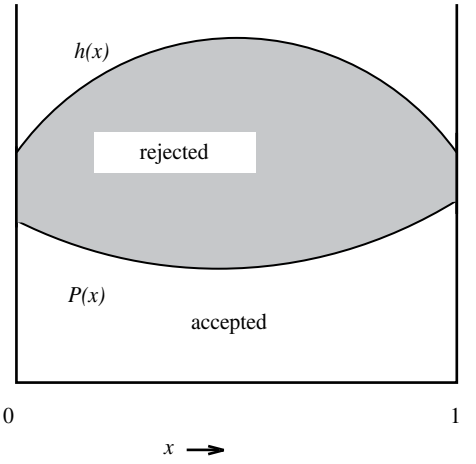


Figure B.1. The von Neumann method for generating nonuniform random numbers.

two numbers v_x and v_y which are both distributed according to a Gaussian. This is called the *Box–Müller method*.

If we cannot find a primitive function for P, we must use a different method. A method by Von Neumann uses at least two uniform random numbers to generate a single nonuniform one. Suppose we want to have a distribution $P(x)$ for x lying between a and b. We start by constructing a generator whose distribution h satisfies $h(x) > \alpha P(x)$ on the interval $[a, b]$. A simple choice is of course the uniform generator, but it is efficient to have a function h with a shape roughly similar to that of P. Now for every x generated by the h-generator, we generate a second random number y uniformly between 0 and 1 and check if y is smaller or larger than $\alpha P(x)/h(x)$. If y is smaller, then x is accepted as the next random number and if y is larger than $\alpha P(x)/h(x)$, it is rejected. The procedure is represented in Figure B.1. Clearly, it is important to have as few rejections as possible, which can be achieved by choosing h such that $\alpha P(x)/h(x)$ is as close as possible to 1 for x between a and b.

Exercises

B.1 Schrage's method [5] enables us to carry out the transformation $ax_n \bmod m$ as it occurs in the modulo random number generator without causing overflow, even when ax_n is greater than the computer's word size. It works as follows. Suppose that for a given a and m, we have two numbers q and r satisfying

$$m = aq + r \quad \text{with } 0 \leq r < q.$$

(a) Show that if $0 < x < m$, both $a(x \bmod q)$ and $r[x/q]$ lie in the range $0, \ldots, m-1$ ($[z]$ is the largest integer smaller than or equal to z).

(b) Using this, show that $ax \bmod m$ can be found as:

$$ax \bmod m = \begin{cases} a(x \bmod q) - r[x/q] & \text{if this is } \geq 0 \\ a(x \bmod q) - r[x/q] + m & \text{otherwise.} \end{cases}$$

(c) Find q and r for $m = 2^{31} - 1$ and $a = 16\,807 (= 7^5)$.

References

[1] N. A. Frigerio and N. Clarck, *Trans. Am. Nucl. Soc.*, **22** (1975), 283–4.

[2] N. A. Frigerio, N. Clarck, and S. Tyler, *Toward Truly Random Random Numbers*, Report, ANL/ES-26 Part 4. Argonne National Laboratory, 1978.

[3] D. E. Knuth, *Seminumerical Algorithms*. *The Art of Computer Programming*, vol. 2. Reading, MA, Addison-Wesley, 1981.

[4] S. W. Park and K. W. Miller, 'Random number generators: good ones are hard to find,' *Commun. ACM*, **31** (1988), 1192–201.

[5] L. Schrage, 'A more portable random number generator,' *ACM Trans. Math. Software*, **5** (1979), 132–8.

[6] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes*, 2nd edn. Cambridge, Cambridge University Press, 1992.

[7] P. l'Ecuyer, 'Efficient and portable combined random number generators,' *Commun. ACM*, **31** (1988), 742–9 and 774.

[8] G. Marsaglia, 'Random numbers fall mainly in the planes,' *Proc. Nat. Acad. Sci.*, **61** (1968), 25–8.

[9] A. M. Ferrenberg, D. P. Landau, and Y. J. Wong, 'Monte Carlo simulations: hidden errors from "good" random number generators,' *Phys. Rev. Lett.*, **69** (1992), 3382–4.

[10] R. C. Tausworthe, 'Random numbers generated by linear recurrence modulo 2,' *Math. Comp.*, **19** (1965), 201–9.

[11] S. Kirkpatrick and E. P. Stoll, 'A very fast shift-register sequence random number generator,' *J. Comp. Phys.*, **40** (1981), 517–26.

Index